
UNIT 5 ARRAY- BASED SEQUENCES

Structure

Page Nos.

5.0	Introduction
5.1	Objectives
5.2	Python's Sequence Types
5.3	Low-Level Arrays
5.4	Dynamic Arrays and Amortization
5.5	Efficiency of Python's Sequences Types
5.6	Using Array-Based Sequences
5.7	Multidimensional Data Sets
5.8	Summary
5.9	Solutions/ Answer to Check your Progress
5.10	References / Further Reading

5.0 INTRODUCTION

Python offers several powerful ways for managing data. Sequence types are fundamental to effective programming, providing developers with flexible tools to handle different kinds of information efficiently. Whether you're working on a small project or a complex application, understanding these sequence types is crucial to writing clean, performant code.

Lists are the most versatile sequence type, allowing you to store and modify collections of items easily. Tuples provide a way to create fixed collections that cannot be changed after creation. Strings handle text data, while ranges generate sequences of numbers efficiently.

We shall look at topics like low-level arrays, dynamic arrays, and multidimensional data structures. These concepts are important for developers who need to optimize memory usage and improve the performance of their Python applications.

This chapter aims to provide a clear, practical understanding of Python's sequence types.

5.1 OBJECTIVES

After going through this unit, you will be able to:

- Define and understand fundamental sequence types in Python
- Compare and contrast different sequence types
- Implement practical operations on sequences
- Choose appropriate sequence types for various programming scenarios

5.2 INTRODUCTION TO SEQUENCE TYPES

Unlike traditional static arrays, Python sequences provide dynamic and flexible

mechanisms for data management.

Sequence types allow for the organization and manipulation of ordered collections of items. Sequences can store multiple items, making them essential for various programming tasks. Sequence maintains elements in order which is accessed through indexes, and the sequence maintains a specific order of elements

We will explore Python's built-in sequence types, including lists, tuples, strings, and ranges. We will discuss their characteristics, operations, and provide practical examples to illustrate their usage.

Fundamental Sequence Types

Python offers three primary built-in sequence types:

1. **Lists:** Mutable sequences that can hold a variety of object types.
2. **Tuples:** Immutable sequences that can also hold a variety of object types.
3. **Strings:** Immutable sequences of characters.
4. **Ranges:** Immutable sequences representing a range of numbers.

Each sequence type possesses unique characteristics and serves distinct programming requirements.

Lists

A list is a mutable ordered collection of items and it supports heterogeneous data types. We can depict lists by []

Characteristics

- **Mutable:** Lists can be modified after their creation.
- **Ordered:** Elements have a defined order that will not change unless explicitly modified.
- **Indexable:** Elements are accessed by their index.

Syntax:

```
# List declaration
```

```
employee_list = ['Ram', 'Ashok', 'Bob']
```

```
mixed_list = [1, 'name', 3.14, True]
```

Examples

```
# An empty list
```

```
empty_list = []
```

```
# A list of integers
```

```
int_list = [1, 2, 3, 4, 5]
```

```
# A list with heterogeneous data types
```

```
mixed_list = [1, "two", 3.0, [4]]
```

```
# A list of strings
string_list = ["apple", "banana", "cherry"]
```

Key Operations:

Creation

Initializing list with elements

Empty list

```
empty_list = []
```

Integers

```
integer_list = [1, 2, 3, 4, 5]
```

List of heterogeneous data types

```
mixed_list = [1, "hello", 3.14, True]
```

Accessing Elements:

Accessing the first element

```
first_element = integer_list[0] # Output: 1
```

Accessing the last data element

```
last_element = integer_list[-1] # Output: 5
```

Slicing a list

```
sublist = integer_list[1:4] # Output: [2, 3, 4]
```

Insertion

Adding elements at specified positions

Appending an element

```
integer_list.append(6)
```

Inserting an element at a specified index

```
integer_list.insert(2, 10)
```

Deletion

Removing elements

```
int_list.remove(2) # Removes the first occurrence of 2
```

Modification

Changing existing elements

```
int_list.append(6) # Adds 6 to the end
```

Traversal

Accessing and iterating through elements

Common List Methods: Given below are common list methods -

- `len(list)`: Returns the number of elements.
- `list.sort()`: Sorts the list in place.
- `list.reverse()`: Reverses the order of elements.

Example -

```
numbers = [5, 1, 4, 2]
numbers.sort()
print(numbers)          # Output: [1, 2, 4, 5]
```

Performance Characteristics:

- Dynamic memory allocation
- Contiguous memory representation
- $O(1)$ access time by index
- $O(n)$ insertion/deletion time complexity deletion, visiting, etc.

Tuples

Tuples provide an immutable alternative to lists, ensuring data integrity and preventing unintended modifications. Tuples are represented using parentheses `()`.

Syntax:

```
# Tuple declaration
coordinates = (10, 20)
employee_record = ('Ashok Kumar', 35, 'Software Engineer')
```

Creating Tuples

```
# Empty tuple
empty_tuple = ()
```

```
# Tuple of integers
integer_tuple = (1, 2, 3)
```

```
# Tuple with a single element (note the comma)
single_element_tuple = (1,)
```

Accessing Elements of Tuple

Accessing elements in a tuple is similar to lists, using indexing and slicing.

Accessing the second element

```
second_element = integer_tuple[1] # Output: 2
```

Slicing a tuple

```
subtuple = integer_tuple[1:] # Output: (2, 3)
```

Key Characteristics:

- Immutability prevents accidental changes
- Lightweight memory representation
- Hashable (can be used as dictionary keys)
- Slightly more memory-efficient than lists

Strings

Strings represent sequences of Unicode characters, fundamental for text processing and manipulation. They are immutable and defined using single quotes " or double quotes ""

Characteristics

- Immutable: Strings cannot be modified after their creation.
- Ordered: Characters have a defined order that will not change.
- Indexable: Characters can be accessed by their index.

Syntax:

String declaration

```
name = "Python Programming"
```

```
multiline_string = """Comprehensive
```

```
Multi-line
```

```
Text"""
```

String Operations

Accessing Characters:

```
print(single_quote_string[0])
```

Slicing Strings

```
print(double_quote_string[1:4])
```

String Methods:

Common string methods include:

- str.lower(): Converts to lowercase.
- str.upper(): Converts to uppercase.
- str.replace(old, new): Replaces occurrences of a substring.

Unique Properties:

- Immutable character sequence
- Rich set of built-in string manipulation methods
- Unicode support
- Efficient memory representation

Ranges

The range() function generates an immutable sequence of numbers commonly used in loops.

Characteristics

- Immutable: Ranges cannot be modified after their creation.
- Ordered: Elements are in a defined numerical order.
- Indexable: Elements can be read by their index.

Creating Ranges

We can create ranges with one to three arguments (start, stop, step):

```
range_1 = range(5)      # Generates numbers from 0 to 4
range_2 = range(1, 10)  # Generates numbers from 1 to 9
range_3 = range(0, 10, 2) # Generates even numbers from 0 to 8
```

Sequence Operations

Common Operations Across Sequence Types:

- Indexing: Accessing elements by position
- Slicing: Extracting subsequences
- Concatenation: Combining sequences
- Repetition: Duplicating sequence elements
- Membership testing
- Length determination

Example:

```
# Demonstration of sequence operations
numbers = [1, 2, 3, 4, 5]
subset = numbers[1:4]    # Slicing
combined = numbers + [6, 7] # Concatenation
repeated = numbers * 2   # Repetition
```

Implementation Considerations

When selecting sequence types, consider:

- Mutability requirements
- Performance needs
- Memory constraints
- Intended usage pattern

Recommendation Matrix:

- Frequent modifications → Use Lists
- Constant data → Use Tuples
- Text processing → Use Strings

Advanced Sequence Techniques

Python provides advanced sequence manipulation techniques:

- List comprehensions
- Generator expressions
- Functional sequence processing methods

Example:

```
# List comprehension  
squared_numbers = [x**2 for x in range(10)]
```

Sequence types form the cornerstone of data representation in Python. Understanding their intrinsic properties, implementation details, and appropriate usage empowers programmers to design elegant and efficient solutions.

5.3 LOW LEVEL ARRAYS

Arrays are crucial for understanding memory management and performance optimization in Python. Low-level arrays manage collection of items allowing for efficient access and manipulation. In Python, low-level arrays differ significantly from high-level lists. While lists are dynamic and can store heterogeneous data types, low-level arrays provide type-constrained, memory-efficient data storage.

An Array is a contiguous memory block storing elements of uniform type in which every element is present a fixed memory location. By leveraging array module and NumPy, developers can create memory-efficient, high-performance applications.

Consider a real-world scenario: developing a scientific simulation requiring precise memory management for large numerical datasets. High-level lists would consume excessive memory, whereas low-level arrays offer compact, efficient storage.

We will discuss the implementation of low-level arrays using the array module in Python, their characteristics, and practical examples.

Low-Level Arrays in Python

Low-level arrays in Python can be implemented using the array module, which provides a space-optimal way to store homogeneous data types. Unlike lists, which can store mixed types of data, arrays require all elements to be of the same type.

Creating Low-Level Arrays

A sample code for array is given below -

```
import array

# Creating an integer array
int_array = array.array('i', [1, 2, 3, 4, 5])

# Creating an array of floats
float_array = array.array('f', [1.0, 2.0, 3.0])
```

In the above examples:

- 'i' denotes an integer type.
- 'f' denotes a float type.
-

Characteristics of Low-Level Arrays

- Homogeneous Elements: All elements in an array must be of the same type.
- Memory Efficiency: Less memory is consumed in arrays as compared to lists.
- Performance: Accessing array elements is faster than in a list due to contiguous memory allocation.

Example:

```
import array

# Create an integer array
numbers = array.array('i', [10, 20, 30, 40])
print(numbers)          # Output: array('i', [10, 20, 30, 40])
```

Array Operations **Insertion**

For inserting elements, we need to shift elements to accommodate the new element. Here's a sample code for insertion is given below:

```
def insert(array_obj, position, value):
    # Insert value at position
    array_obj.insert(position, value)

# Example usage
numbers = array.array('i', [10, 20, 30])
insert(numbers, 1, 15)  # Insert '15' at index '1'
print(numbers)          # Output: array('i', [10, 15, 20, 30])
```

Deletion

A sample code to delete the item at a specified position is given below:

```
def delete(array_obj, position):
    # Delete value at position
    del array_obj[position]

# Example usage
```

```
numbers = array.array('i', [10, 15, 20, 30])
delete(numbers, 1)      # Delete element at index '1'
print(numbers)          # Output: array('i', [10, 20, 30])
```

Traversal

Array traversal is sequential as shown below -

```
def traverse(array_obj):
    for i in range(len(array_obj)):
        print(array_obj[i], end=' ')

# Example usage
numbers = array.array('i', [10, 20, 30])
traverse(numbers)      # Output: 10 20 30
```

Performance Considerations

Low-level arrays offer significant performance advantages:

- Reduced memory overhead
- Faster computational operations
- Type-specific optimizations
- Direct memory manipulation
-

Practical Applications

Low-level arrays find applications in:

- Scientific computing
- Numerical simulations
- Data science: Data analysis, machine learning, and statistical modeling
- Image and signal processing: Image filtering, feature extraction, and signal analysis
- High-performance computing
- Embedded systems programming

5.4 DYNAMIC ARRAYS AND AMORTIZATION

Dynamic Arrays in Python

Dynamic arrays expand and shrink in size as elements are added or removed. In Python, lists serve as dynamic arrays, but understanding their underlying mechanics provides insight into their efficiency.

A dynamic array allows elements to be added or removed dynamically, with automatic memory reallocation. In Python, lists serve as the primary implementation of dynamic arrays. Mathematically, a dynamic array is an expandable element sequence with $O(1)$ amortized time complexity for deletion and insert operations. When the array reaches its capacity, a new array of larger capacity is allocated, and the elements from the old array are copied to the new one. Additional elements are accommodated due to the resizing process.

Consider a real-world scenario where we need to store student records in a growing or shrinking collection. Traditional static arrays would require manual

memory management, while Python's dynamic arrays handle this transparently.

Creating Dynamic Arrays

In Python, a dynamic array can be created simply by initializing a list. Unlike static arrays, which require a predefined size, lists automatically resize when elements are added or removed.

```
python
# Creating a dynamic array (list)
dynamic_array = []
```

Characteristics of Dynamic Arrays

- **Resizable:** Dynamic arrays can grow or shrink as needed.
- **Homogeneous Elements:** While lists can hold mixed data types, the underlying implementation typically uses homogeneous types for efficiency.
- **Amortized Cost:** The average time complexity for appending an element is $O(1)$ but resizing incurs a cost.

Example:

```
# Adding elements to a dynamic array
dynamic_array.append(10)
dynamic_array.append(20)
dynamic_array.append(30)
print(dynamic_array) # Output: [10, 20, 30]
```

Dynamic Array Resizing

When a dynamic array reaches its capacity, it must resize to accommodate new elements. This involves creating a new larger array and copying the existing elements into it.

Resizing Process

1. **Allocation:** When the current capacity is reached, a new array of double the size is created.
2. **Copying:** Existing elements are copied from the old array to the new array.
3. **Deallocation:** The old array is deallocated (if applicable).

Example:

```
import sys

def create_dynamic_array(initial_size):
    return [None] * initial_size

def append(array, size, value):
    if size == len(array): # Check if resizing is needed
        new_array = create_dynamic_array(len(array) * 2) # Double the size
        for i in range(size):
            new_array[i] = array[i] # Copy elements
        array = new_array
        array[size] = value
```

```

return array, size + 1

# Example usage
dynamic_array = create_dynamic_array(2)
size = 0

for value in [10, 20, 30]:
    dynamic_array, size = append(dynamic_array, size, value)

print(dynamic_array) # Output: [10, 20]

```

Amortization In Dynamic Arrays

The concept of amortization helps analyze the average time complexity of operations over a sequence of actions rather than individual actions. In the context of dynamic arrays:

- **Amortized Time Complexity:** Although resizing an array takes $O(n)$ time when it occurs, the average time per insertion remains $O(1)$ due to infrequent resizing.

Example Analysis:

Suppose we perform n insertions into a dynamic array:

1. The first insertion takes $O(1)$.
2. The second insertion takes $O(1)$.
3. The third insertion might trigger a resize and take $O(n)$.

However, if we analyze across all insertions:

- Most insertions are $O(1)$.
- Resizing occurs logarithmically relative to the number of insertions.

Thus, the average cost per insertion remains constant at $O(1)$.

Practical Considerations

Understanding dynamic array behavior helps in:

- Efficient memory utilization
- Predicting performance characteristics
- Designing scalable data structures

5.5 EFFICIENCY OF PYTHON'S SEQUENCETYPES

This chapter focuses on the performance aspects of Python's sequence types, examining their time and space complexities, and providing insights into when to use each type effectively.

Time Complexity Of Sequence Types

Lists

Python lists are dynamic arrays that allow for efficient element access and modification. The following table summarizes the time complexities for common operations on lists:

Operation	Time Complexity
-----------	-----------------

Operation	Time Complexity
Access by index	O(1)
Append	O(1)
Insert at beginning	O(n)
Insert at end	O(1)
Remove from end	O(1)
Remove from beginning	O(n)

Example:

python

Accessing an element

`my_list = [10, 20, 30, 40]`

`print(my_list[2])` *# Output: 30*

Appending an element

`my_list.append(50)`

`print(my_list)` *# Output: [10, 20, 30, 40, 50]*

Inserting at the beginning

`my_list.insert(0, 5)`

`print(my_list)` *# Output: [5, 10, 20, 30, 40, 50]*

Tuples

Tuples are immutable sequences that offer several performance advantages because of their fixed size. The time complexities for tuple operations are as follows:

Operation	Time Complexity
Access by index	O(1)
Search	O(n)

Example:

Python

Creating a tuple

`my_tuple = (10, 20, 30)`

Accessing an element

`print(my_tuple[1])` *# Output: 20*

Strings

Strings in Python are immutable sequences of characters. The time

complexities for string operations include:

Operation	Time Complexity
Access by index	$O(1)$
Concatenation	$O(n)$
Search	$O(n)$

Example:

```
python
```

```
# Creating a string
```

```
my_string = "Hello"
```

```
# Accessing a character
```

```
print(my_string[1]) # Output: e
```

```
# Concatenating strings
```

```
new_string = my_string + " World"
```

```
print(new_string) # Output: Hello World
```

Ranges

Ranges are used to represent a sequence of numbers and are particularly memory-efficient. Their time complexities are as follows:

Operation	Time Complexity
Access by index	$O(1)$
Length retrieval	$O(1)$

Example:

```
python
```

```
# Creating a range
```

```
my_range = range(5)
```

```
# Accessing an element
```

```
print(my_range[2]) # Output: 2
```

Space complexity of sequence types

Lists

Lists consume more memory than arrays due to their dynamic nature and overhead associated with maintaining references to elements.

Tuples

Tuples are more memory-efficient than lists because they no additional space is required for dynamic resizing.

Strings

Strings also consume more memory due to their immutability and the need for additional storage when concatenating or modifying.

Ranges

Ranges are highly space-efficient as they do not store all values in memory but instead generate them on demand.

When To Use Each Sequence Type

Understanding the efficiency of each sequence type allows developers to make informed decisions based on their specific needs:

- Use lists when you require a mutable sequence with fast appends and random access. Lists can be used for Dynamic collections with Frequent insertions/deletions and for handling Heterogeneous data
- Use tuples when you need an immutable sequence that can serve as keys in dictionaries or when you want to ensure data integrity. We can leverage tuples for fixed collections when we need Performance-critical code and for handling Dictionary keys
- Use strings for text manipulation where immutability is required.
- Use ranges when working with sequences of numbers without needing to store them explicitly.

Comparative Efficiency

To demonstrate efficiency differences, consider the following performance characteristics:

1. Lists: • Mutable • Dynamic resizing • Higher memory overhead • Flexible but slower for large datasets
2. Tuples: • Immutable • Fixed size • Lower memory overhead • Faster creation and access
3. Arrays (using numpy): • Homogeneous data • Contiguous memory • Fastest numeric operations • Less flexible than lists

5.6 USING ARRAY BASED SEQUENCES

Python, a versatile programming language, offers robust support for array-based sequences through its built-in list type and the powerful NumPy library. This chapter delves into the intricacies of using array-based sequences in Python, exploring their underlying principles, common operations, and real-world applications.

Python's Built-in List

Python's list type is a highly flexible data structure for representing array-based sequences. It provides a dynamic and efficient way to store and manipulate collections of elements.

Creating and Accessing Lists

Lists can be created using square brackets [] and populated with elements of various data types:

```
my_list = [1, 2, 3, "hello", 4.5]
```

Elements within a list can be accessed using zero-based indexing:

```
first_element = my_list[0] # Accesses the first element
```

```
last_element = my_list[-1] # Accesses the last element
```

List Operations

Python's list type offers a rich set of operations for manipulating sequences:

Appending Elements:

```
my_list.append(6) # Adds 6 to the end of the list
```

Inserting Elements:

```
my_list.insert(2, "world") # Inserts "world" at index 2
```

Removing Elements:

```
my_list.remove("hello") # Removes the first occurrence of "hello"
```

Slicing:

```
sublist = my_list[1:4] # Extracts a slice from index 1 to 3
```

Iterating:

```
for element in my_list:  
    print(element)
```

NumPy Arrays

For numerical computations and scientific applications, NumPy provides a powerful array object that is optimized for performance and efficiency. NumPy arrays store elements of the same data type.

Creating NumPy Arrays

A sample code for NumPy arrays is given below -

```
import numpy as np
```

```
my_array = np.array([1, 2, 3, 4, 5])
```

NumPy Array Operations

NumPy arrays support a wide range of operations, including:

- Arithmetic Operations:

Python

```
array1 = np.array([1, 2, 3])
```

```
array2 = np.array([4, 5, 6])
```

```
result = array1 + array2 # Element-wise addition
```

Matrix Operations:

Python

```
matrix1 = np.array([[1, 2], [3, 4]])
```

```
matrix2 = np.array([[5, 6], [7, 8]])
```

```
product = np.dot(matrix1, matrix2) # Matrix multiplication
```

Universal Functions

NumPy provides a rich set of universal functions (ufuncs) for element-wise operations on arrays.

Applications of Array-Based Sequences

Array-based sequences are fundamental to a wide range of applications, including:

- Data Analysis and Scientific Computing:
 - Storing and processing large datasets
 - Performing numerical simulations and statistical analysis
- Image and Signal Processing:
 - Representing and manipulating images and signals
- Machine Learning:
 - Training and deploying machine learning models
- Web Development:
 - Handling and processing data in web applications

5.7 MULTI-DIMENSIONAL DATA SET

In the previous sections, we explored the concept of one-dimensional data structures, such as lists and arrays. However, in many real-world applications, data is often multidimensional, requiring more complex structures to represent it efficiently. In this chapter, we will delve into multidimensional data sets in Python, focusing on their implementation using lists and the powerful NumPy library.

Multidimensional Data Structures

Multidimensional data structures allow us to store data in more than one dimension. The most common type is the matrix, which is a two-dimensional array. In Python, we can represent multidimensional data using nested lists or by leveraging libraries like NumPy that provide optimized array operations.

Nested Lists

A nested list is a list that contains other lists as its elements. This structure can be used to create a two-dimensional array.

Example: Creating a Nested List

Creating a 2D nested list (matrix)

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

In this example, matrix is a two-dimensional list containing three rows and three columns.

Manipulating Multidimensional Lists

Manipulating multidimensional lists involves accessing and modifying elements based on their indices.

Accessing Elements

To access an element in a nested list, we use two indices: one for the row and one for the column.

Example: Accessing an Element

```
# Accessing an element at row 1, column 2
element = matrix[1][2] # This will return 6
```

Modifying Elements

We can modify elements by assigning new values to specific indices.

Example: Modifying an Element

```
# Changing the value at row 0, column 1
matrix[0][1] = 20 # Now matrix becomes [[1, 20, 3], [4, 5, 6], [7, 8, 9]]
```

Using Numpy For Multidimensional Data

Nested lists face challenges for larger datasets or more complex operations.

The NumPy library provides a powerful alternative with its ndarray object designed for efficient numerical computations.

Creating NumPy Arrays

To use NumPy for multidimensional data sets, we first need to install it (if not already installed) and then import it into our Python script. Installation

Command:

```
bash
pip install numpy
#Creating a NumPy Array
```

```
import numpy as np
# Creating a 2D NumPy array (matrix)
matrix_np = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Accessing and Modifying NumPy Arrays

Accessing and modifying elements in a NumPy array is similar to that of nested lists but with additional functionalities.

Accessing Elements

```
python
# Accessing an element at row 1, column 2
element_np = matrix_np[1][2] # This will return 6
```

Modifying Elements

```
python
# Changing the value at row 0, column 1
matrix_np[0][1] = 20 # Now matrix_np becomes [[1, 20, 3], [4, 5, 6], [7, 8, 9]]
```

Applications Of Multidimensional Data Structures

Given below are few use cases for multidimensional data structures -

- Data Analysis: Storing datasets with multiple features (e.g., images with pixel values).
- Scientific Computing: Representing matrices in linear algebra.
- Machine Learning: Handling datasets with multiple attributes for training algorithms

CheckYourProgress 1

- 1) Immutable sequences of characters is called _____
- 2) _____ can be used for frequent modifications
- 3) _____ is the best choice for constant data
- 4) All elements in the array should be homogenous True/False
- 5) The average cost per insertion in dynamic array is _____
- 6) Time complexity for "Remove from beginning" operation for Lists is _____

5.8 SUMMARY

1. Fundamental Sequence Types Python offers four primary sequence types:

- Lists (Mutable)
- Tuples (Immutable)
- Strings (Immutable)
- Ranges (Immutable)

2. Lists

- Dynamic, mutable sequences
- Can store mixed data types
- Flexible for modifications
- Key operations: indexing, slicing, appending, inserting, deleting
- Efficient for frequent changes

3. Tuples

- Immutable sequences
- Lightweight memory representation
- Hashable (can be used as dictionary keys)
- More memory-efficient than lists

4. Strings

- Immutable sequences of Unicode characters
- Rich set of string manipulation methods
- Used for text processing

5. Ranges

- Generates sequences of numbers
- Memory-efficient
- For Loops

6. Low-Level Arrays

- Implemented using the array module

- Homogeneous data types
 - More memory-efficient than lists
 - Used in scientific computing and high-performance applications
7. Dynamic Arrays
- Can grow and shrink dynamically
 - Amortized $O(1)$ insertion time
 - Automatic memory reallocation
 - Resizing involves creating a new, larger array and copying elements
8. Efficiency Considerations Performance characteristics vary:
- Lists: Flexible, good for dynamic collections
 - Tuples: Fast access, immutable
 - Strings: Good for text manipulation
 - Ranges: Most memory-efficient
9. Multidimensional Data Supported through:
- Nested lists
 - NumPy arrays (more efficient for numerical computations)
- Key Takeaways:
- Choose sequence type based on specific requirements
 - Consider mutability, performance, and memory constraints
 - NumPy can be used for numerical and scientific computing

5.9 SOLUTIONS/ANSWERS

- 1) Strings
- 2) Lists
- 3) Tuples
- 4) True
- 5) $O(1)$
- 6) $O(n)$

5.10 FURTHER READINGS

- 1) "Python for Data Analysis" by Wes McKinney
- 2) "Python Data Science Handbook" by Jake VanderPlas
- 3) "Learning NumPy Array" by Ivan Idris
- 4) "Data Science from Scratch: First Principles with Python" by Joel Grus
- 5) Online Documentation and Tutorials
 - NumPy Documentation:
 - pandas Documentation

UNIT 6 STACKS, QUEUES, AND DEQUES

Structure

Page Nos.

6.0	Introduction
6.1	Objectives
6.2	Abstract Data Type-List
6.3	Array Implementation of Lists
6.4	Linked Lists-Implementation
6.5	Doubly Linked Lists-Implementation
6.6	Circularly Linked Lists-Implementation
6.7	Applications
6.8	Summary
6.9	Solutions/Answers
6.10	Further Readings

6.0 INTRODUCTION

In computer science, data structures are essential for organizing and managing data efficiently. Among the various types of data structures, **stacks**, **queues**, and **double-ended queues (dequeues)** are fundamental concepts that enable developers to handle collections of data in specific ways.

Stacks operate on a Last In, First Out (LIFO) basis, where the most recently added element are removed first. This structure is analogous to a stack of plates; you can only add or remove the top plate. We use Stacks for expression evaluation, backtracking and for function call management.

Queues, on the other hand, follow a First In, First Out (FIFO) principle. This means first added element to the queue will be the first one to be removed, similar to a line of customers at a service counter. Queues are widely utilized in scheduling tasks, managing requests in web servers, and handling asynchronous data processing.

Double-ended queues (dequeues) provide the flexibility of insertion and removal from both ends. This allows for operations similar to both stacks and queues. Deques are particularly useful in scenarios where elements need to be processed from both ends efficiently, such as in certain algorithms for breadth-first search or when implementing undo mechanisms in applications. Understanding these data structures is crucial for effective programming and algorithm design. They provide the foundational tools needed to manipulate collections of data efficiently and can significantly impact the performance of applications.

6.1 OBJECTIVES

After completing this unit, you will be able to:

- Define and differentiate between stacks, queues, and dequeues
- Implement basic operations for each data structure in Python
- Analyse the time complexity of operations in these structures

- Choose the best data structure for specific use cases
- Apply these data structures to solve real-world problems
- Understand and implement common variations of these structures

6.2 INTRODUCTION TO STACKS

A stack is one of the most fundamental abstract data structures in computer programming. We add and remove the elements from the same end of the stack which follows Last-in-first-out (LIFO) principle. Understanding stacks is crucial for many programming applications, from managing program execution to implementing undo operations in applications.

This chapter explores the implementation, operations, and practical applications of stacks in Python, providing you with the knowledge to effectively use this important data structure in your programming projects.

Understanding Stacks

A stack can be visualized as a vertical list of elements. The operations associated with stacks are primarily:

- **Push:** Adding an element to the top of the stack.
- **Pop:** Removing the element from the top of the stack.
- **Peek:** Retrieving the top element without removing it.

Below given diagram shows the push and pop operations

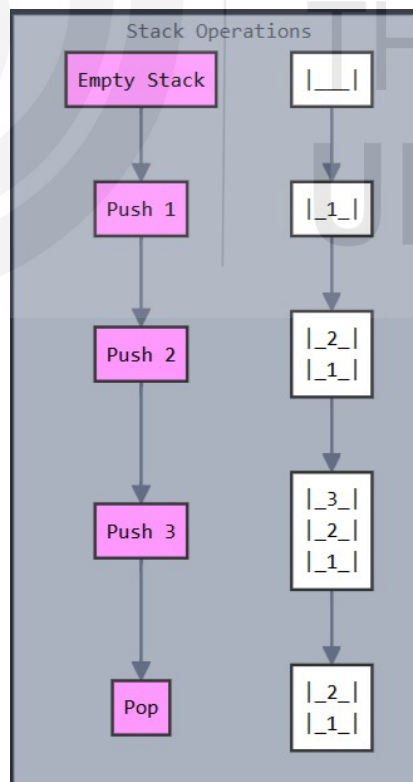


Figure 6.1 : Stack Operations

Characteristics of Stacks

- **LIFO Principle:** The last element added is removed first
- **Dynamic Size:** Stacks grow when we add or delete elements.
- **Limited Access:** Elements can only be accessed from stack's top.

Implementing a Stack in Python

In Python, stacks can be easily implemented using lists or the collections.deque module. Below are examples of both implementations.

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        raise IndexError("pop from empty stack")

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        raise IndexError("peek from empty stack")

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

# Example usage
stack = Stack()
stack.push(1)
stack.push(2)
print(stack.peek()) # Output: 2
print(stack.pop())  # Output: 2
print(stack.size()) # Output: 1
```

Using collections.deque

The deque class from the collections module provides an efficient way to implement stacks due to its optimized performance for appending and popping elements.

```
from collections import deque
```

```
class Stack:
```

```
    def __init__(self):
```

```
        self.items = deque()
```

```
    def push(self, item):
```

```
        self.items.append(item)
```

```
    def pop(self):
```

```
        if not self.is_empty():
```

```
            return self.items.pop()
```

```
        raise IndexError("pop from empty stack")
```

```
    def peek(self):
```

```
        if not self.is_empty():
```

```
            return self.items[-1]
```

```
        raise IndexError("peek from empty stack")
```

```
    def is_empty(self):
```

```
        return len(self.items) == 0
```

```
    def size(self):
```

```
        return len(self.items)
```

```
# Example usage
```

```
stack = Stack()
```

```
stack.push(3)
```

```
stack.push(4)
```

```
print(stack.peek()) # Output: 4
```

```
print(stack.pop()) # Output: 4
```

```
print(stack.size()) # Output: 1
```

Key Operations

1. Push Operation:

- Adds an element to the top of the stack.
- Time Complexity: $O(1)$

2. Pop Operation:

- Removes and returns the top element of the stack.
- Time Complexity: $O(1)$

3. Peek Operation:

- Returns the top element without removing it.
- Time Complexity: $O(1)$

4. Is Empty Check:

- Checks if the stack is empty.
- Time Complexity: $O(1)$

5. Size Check:

- Returns the number of elements in the stack.
- Time Complexity: $O(1)$

Performance Considerations

Different stack implementations offer varying performance characteristics:

Operation	List based Stack	Deque-based stack
Push	$O(1)$	$O(1)$
Pop	$O(1)$	$O(1)$
peek	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$

Best Practices

- Use `collections.deque` for better performance when implementing stacks due to its optimized append and pop operations.
- Always check for underflow conditions (attempting to pop from an empty stack) to avoid runtime errors.
- Consider encapsulating stack operations within a class to maintain data integrity and provide a clear interface.

Practical Use Cases

Stacks have numerous applications in computer science and programming:

- **Function Call Management:** We keep track of active functions and program execution variables in the call stack
- **Expression Evaluation:** Stacks are used in parsing expressions (e.g., infix to postfix expressions).

- **Backtracking Algorithms:** Algorithms like depth-first search (DFS) utilize stacks to explore nodes in graphs or trees efficiently.
- **Undo Mechanisms:** The word editors that handle undo operations use stack for managing the earlier states.

Understanding stacks is essential for effective programming and algorithm design. Stacks are used in various use cases. By mastering stack operations and their implementations in Python, developers can leverage this powerful data structure in their coding

6.3 QUEUES

In Python programming, queues store and manipulate elements in a FIFO (First-In-First-Out) manner. Unlike static data structures, queues provide dynamic and flexible mechanisms for managing ordered collections where the first element is the first one to be removed. Queues are essential in various programming scenarios, particularly in managing tasks, scheduling processes, and handling asynchronous data.

In this section, we will discuss the queue data structure in Python, covering its implementation, key operations, best practices, and practical use cases.

Understanding Queues

A queue can be visualized as a collection of elements arranged in a linear order. The operations associated with queues include:

- **Enqueue:** Adds an element to the back of the queue.
- **Dequeue:** Removing an element from the queue front
- **Peek:** Retrieving the front element without removing it.
- **IsEmpty:** Checks the emptiness of the queue
- **Size:** Returns the number of elements in the queue

Below given diagram shows Enqueue and Dequeue operations –

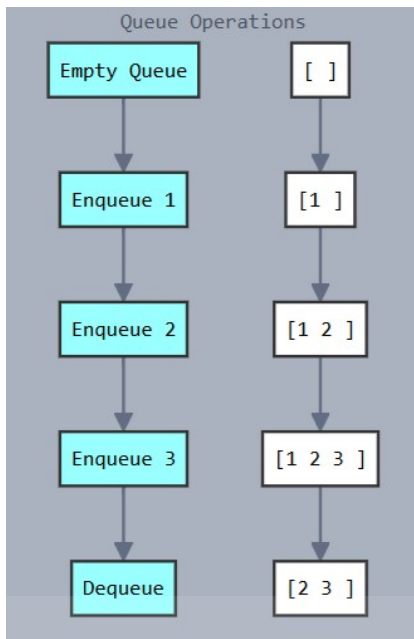


Figure 6.2 : Queue Operations

Fundamental Queue Types

Python offers several ways to implement queues:

1. **List-based Queues:** Simple way to impliment the queues.
2. **collections.deque:** Optimized for queue operations
3. **queue.Queue:** Thread-safe implementation for concurrent programming
4. **Priority Queues:** Special queues where elements have priorities

Characteristics of Queues

- FIFO Principle: The first element added is the first to be removed.
- Dynamic Size: Queues can grow or shrink as elements are added or removed.
- Limited Access: Elements can only be accessed from the queue front.

Implementing a Queue in Python

In Python, queues can be implemented using lists or the 'collections.deque' module. Below are examples of both implementations.

Using Lists

```

class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        raise IndexError("dequeue from empty queue")

```

```

def peek(self):
    if not self.is_empty():
        return self.items[0]
    raise IndexError("peek from empty queue")

def is_empty(self):
    return len(self.items) == 0

def size(self):
    return len(self.items)

# Example usage
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print(queue.peek()) # Output: 1
print(queue.dequeue()) # Output: 1
print(queue.size()) # Output: 1
'''

```

Using `collections.deque`

The `deque` class from the `collections` module provides an efficient way to implement queues due to its optimized performance for appending and popping elements from both ends.

```

from collections import deque

class Queue:
    def __init__(self):
        self.items = deque()

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.popleft()
        raise IndexError("dequeue from empty queue")

    def peek(self):
        if not self.is_empty():
            return self.items[0]
        raise IndexError("peek from empty queue")

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

# Example usage

```

```

queue = Queue()
queue.enqueue(3)
queue.enqueue(4)
print(queue.peek()) # Output: 3
print(queue.dequeue()) # Output: 3
print(queue.size()) # Output: 1
'''

```

Key Operations

1. Enqueue Operation:

- Adds an element to the back of the queue.
- Time Complexity: $O(1)$ when using `collections.deque`, $O(n)$ when using lists due to shifting elements.

2. Dequeue Operation:

- Removes and returns the front element.
- Time Complexity: $O(1)$ when using `collections.deque`, $O(n)$ when using lists.

3. Peek Operation:

- Returns the front element without removing it.
- Time Complexity: $O(1)$

4. Is Empty Check:

- Checks if the queue is empty.
- Time Complexity: $O(1)$

5. Size Check:

- Returns the number of elements in the queue.
- Time Complexity: $O(1)$

Performance Characteristics

Given below are performance characteristics of various queue implementations–

Operation	List-based Queue	deque-based Queue	Queue-based Queue	Priority Queue
Enqueue	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
Front	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Size	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Best Practices

Given below are the main best practices:

1. Choose the Right Implementation

- Use `collections.deque` for general-purpose queues
- Use `Queue.Queue` for thread-safe operations

- Use PriorityQueue when elements have priorities
- 2. **Exception Handling**
 - Always handle queue underflow and overflow conditions
 - Implement proper error messages for exceptional cases
- 3. **Memory Management**
 - Consider implementing maximum size limits
 - Clear queues when no longer needed
- 4. **Performance Optimization**
 - Use deque for better performance in large queues
 - Avoid list-based implementation for large datasets

Practical Use Cases

Queues have numerous applications in computer science and programming:

- Task Scheduling: Operating systems use queues to manage processes and scheduling tasks efficiently.
- Breadth-First Search (BFS): Queues are used in in BFS based graph traversal to explore nodes level by level.
- Print Queue Management: Printers manage print jobs using queues to ensure that documents are printed in the order they were received.
- Asynchronous Data Handling: Queues handle communication across multithreaded environments.

Let us look at the code for queues for task scheduling as given below -

```
class TaskScheduler:
    def __init__(self):
        self.task_queue = Queue()

    def add_task(self, task):
        self.task_queue.enqueue(task)

    def process_tasks(self):
        while not self.task_queue.is_empty():
            task = self.task_queue.dequeue()
            print(f"Processing task: {task}")
            # Execute task logic here

# Example usage
scheduler = TaskScheduler()
scheduler.add_task("Send email")
scheduler.add_task("Generate report")
scheduler.add_task("Backup database")
scheduler.process_tasks()
```

Another example of using Queue for print job as given below -

```
class PrintQueue:
    def __init__(self):
        self.print_queue = PriorityQueue()
```

```
def add_document(self, document, priority):
    self.print_queue.enqueue(document, priority)

def process_print_jobs(self):
    while not self.print_queue.is_empty():
        document = self.print_queue.dequeue()
        print(f"Printing: {document}")
```

Understanding queues is crucial for effective programming and algorithm design. By mastering queue operations and their implementations in Python, developers can leverage this powerful data structure in their coding practices effectively.

6.4 DOUBLE ENDED QUEUES

A double-ended queue, commonly referred to as a deque, allows insertion and removal of elements from both ends, making it a hybrid between stacks and queues. This flexibility enables efficient operations at both the front and the back of the structure, making deques suitable for a variety of applications in programming.

In this section, we will explore the deque data structure in Python, including its implementation, key operations, best practices, and practical use cases.

Understanding Deques

A deque can be visualized as a linear collection of elements where you can add or remove items from both ends. The primary operations associated with deques include:

- Append: Adding an element to the back of the deque.
- Appendleft: Adding an element to the front of the deque.
- Pop: Removing an element from the back of the deque.
- Popleft: Removing an element from the front of the deque.
- Peek: Retrieving the front or back element without removing it.

Below given diagram shows append, peek, pop operations in dequeue -

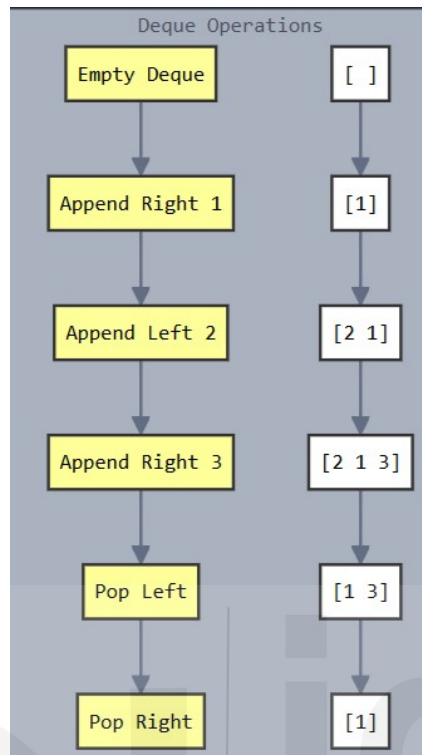


Figure 6.3 : Deque Operations

Characteristics of Deques

- **Double-Ended:** Elements can be added or removed from either end
- **Thread-Safe:** Safe for concurrent access from multiple threads
- **Memory-Efficient:** Optimized memory usage compared to lists
- **Fast Operations:** $O(1)$ complexity for append and pop operations at both ends
- **Flexible Operations:** Elements can be added or removed from both ends.
- **Dynamic Size:** Deques can grow or shrink as elements are added or removed.
- **Efficient Access:** Fast access to both ends with constant time complexity for pop and append operations.

Implementing a Deque in Python

In Python, `'collections.deque'` class is used for implementing deque, which provides an efficient way to manage double-ended queues.

```
# Using 'collections.deque'
```

```
from collections import deque
```

```
class Deque:
    def __init__(self):
        self.items = deque()
```

```
def append(self, item):
    self.items.append(item)
```

```
def appendleft(self, item):
    self.items.appendleft(item)
```

```
def pop(self):
    if not self.is_empty():
        return self.items.pop()
    raise IndexError("pop from empty deque")
```

```
def popleft(self):
    if not self.is_empty():
        return self.items.popleft()
    raise IndexError("popleft from empty deque")
```

```
def peek(self):
    if not self.is_empty():
        return self.items[0]
    raise IndexError("peek from empty deque")
```

```
def is_empty(self):
    return len(self.items) == 0
```

```
def size(self):
    return len(self.items)
```

```
# Example usage
deque = Deque()
deque.append(1)
deque.append(2)
deque.appendleft(0)
print(deque.peek()) # Output: 0
print(deque.pop())  # Output: 2
print(deque.popleft()) # Output: 0
print(deque.size()) # Output: 1
'''
```

Key Operations

1. Append Operation:

- Adds an element to the back of the deque.
- Time Complexity: $O(1)$

Adding single elements

```
my_deque = deque()
```

```
my_deque.append(10)    # Adds to right
```

```
my_deque.appendleft(5) # Adds to left
```

Adding multiple elements

```
my_deque.extend([20, 30]) # Extends on right
```

```
my_deque.extendleft([3, 2, 1]) # Extends on left (note: elements are added in reverse)
```

2. Appendleft Operation:

- Adds an element to the front of the deque.
- Time Complexity: $O(1)$

3. Pop Operation:

- Removes the back element of the deque.
- Time Complexity: $O(1)$

Basic removal

```
right_val = my_deque.pop()
```

```
left_val = my_deque.popleft()
```

Clearing the deque

```
my_deque.clear()
```

4. Popleft Operation:

- Removes the front element of the deque.
- Time Complexity: $O(1)$

5. Peek Operation:

- Returns the front element without removing it.
- Time Complexity: $O(1)$

6. Is Empty Check:

- Checks if the deque is empty.
- Time Complexity: $O(1)$

7. Size Check:

- Returns the deque element size.
- Time Complexity: $O(1)$

Performance Characteristics

Operation	Deque	List
Append/Pop at end	$O(1)$	$O(1)$
Append/Pop at beginning	$O(1)$	$O(n)$
Random Access	$O(n)$	$O(1)$
Length Check	$O(1)$	$O(1)$
Memory Usage	Lower	Higher

Best Practices

- Utilize `'collections.deque'` for implementing double-ended queues due to its optimized performance for appending and popping elements at both ends.
- Always check for underflow conditions (attempting to pop or popleft from an empty deque) to avoid runtime errors.
- Encapsulate deque operations within a class to maintain data integrity and provide a clear interface.

When to use Dequeues

- Use deques when you need:
 - Fast append/pop operations at both ends
 - Thread-safe operations
 - Memory-efficient storage
 - Rotation operations
- Prefer lists when you need:
 - Random access to elements
 - Slice operations
 - Memory is not a constraint

Practical Use Cases

Deques have numerous applications in computer science and programming:

- Task Scheduling: Deques can efficiently manage tasks that require processing from both ends, such as in breadth-first search algorithms where nodes are added and removed dynamically.
- Sliding Window Problems: Deques are often used in algorithms that require maintaining a window of elements for computations like maximum or minimum values over a sliding range.

- Undo Mechanisms: Applications such as text editors can utilize deques to manage undo and redo operations by keeping track of changes at both ends.
- Buffer Management: Deques can serve as buffers that handle streaming data efficiently by allowing insertion and removal of data packets at both ends.

Sample code for sliding window

```
from collections import deque
```

```
def sliding_window_max(nums, k):
    """Find maximum elements in sliding windows of size k."""
    result = []
    window = deque()

    for i, num in enumerate(nums):
        # Remove elements outside current window
        while window and window[0] < i - k + 1:
            window.popleft()

        # Remove smaller elements
        while window and nums[window[-1]] < num:
            window.pop()

        window.append(i)

        if i >= k - 1:
            result.append(nums[window[0]])

    return result

# Example usage
nums = [1, 3, -1, -3, 5, 3, 6, 7]

k = 3
print(sliding_window_max(nums, k)) # Output: [3, 3, 5, 5, 6, 7]
```

Sample code for history tracking

```
class BrowserHistory:
    def __init__(self):
        self.history = deque()
        self.current_index = -1

    def visit(self, url):
```

```

    # Remove forward history
    while len(self.history) > self.current_index + 1:
self.history.pop()
self.history.append(url)
self.current_index += 1

    def back(self, steps):
        steps = min(steps, self.current_index)
self.current_index -= steps
        return self.history[self.current_index]

    def forward(self, steps):
remaining_steps = len(self.history) - self.current_index - 1
        steps = min(steps, remaining_steps)
self.current_index += steps
        return self.history[self.current_index]

```

Implementation Guidelines

1. Size Management

```

# Creating a deque with maximum size
limited_deque = deque(maxlen=3)
for i in range(5):
    limited_deque.append(i)
print(limited_deque) # Output: deque([2, 3, 4], maxlen=3)

```

2. Thread Safety

```

from collections import deque
from threading import Thread

```

```

shared_deque = deque()

```

```

def worker():
    for i in range(1000):
shared_deque.append(i)
        if shared_deque:
shared_deque.pop()

```

```

threads = [Thread(target=worker) for _ in range(4)]
for t in threads:
    t.start()

```

Understanding double-ended queues is essential for effective programming and algorithm design. Their flexibility in managing elements at both ends can be used for implementing various applications across different domains. By mastering deque operations and their implementations in Python, developers can leverage this powerful data structure in their coding practices effectively.

Check your progress 1

1. What principle does a stack follow? a) FIFO b) LIFO c) Random Access d) Sequential Access

2. In a queue, if we remove an element, from which end does it happen? a) Front b) Rear c) Middle d) Any position
3. Which operation would you use to add an element to a stack? a) enqueue b) push c) insert d) add
4. A dequeue allows operations on: a) Only front end b) Only rear end c) Both ends d) Middle only
5. What is the time complexity of push operation in a stack? a) $O(n)$ b) $O(1)$ c) $O(\log n)$ d) $O(n^2)$
6. True or False: A queue can be implemented using two stacks.
7. Which of these applications would best use a stack? a) Print queue in an operating system b) Undo operation in a text editor c) Breadth-first search in a graph d) Round-robin scheduling
8. What happens when you try to pop from an empty stack? a) Returns null b) Throws an exception c) Returns -1 d) Depends on implementation.

6.5 SOLUTIONS/ANSWERS TO CHECK YOUR PROGRESS

1. Answer: b) LIFO
2. Answer: a) Front
3. Answer: b) push
4. Answer: c) Both ends
5. Answer: b) $O(1)$
6. Answer: True
7. Answer: b) Undo operation in a text editor
8. Answer: b) Throws an exception

6.6 SUMMARY

Stacks, queues, and dequeues are fundamental data structures that are used in computer programming:

Stacks (LIFO):

- Support push and pop operations
- Ideal for tracking state and handling recursion
- Common applications include expression evaluation and backtracking
- $O(1)$ is the basic operations time complexity

Queues (FIFO):

- Support enqueue and dequeue operations
- Perfect for scheduling and buffering
- Used in breadth-first search and print spooling
- $O(1)$ is the basic operations time complexity

Dequeues:

- Combine features of stacks and queues
- Allow insertion and deletion at both ends

- Useful for sliding window problems and palindrome checking
- Provide flexibility with same performance benefits

Key Implementation Considerations:

- Can be implemented using arrays or linked lists
- Array implementations may require resizing
- Linked list implementations offer dynamic memory usage
- Choice depends on specific use case requirements

These data structures are the basic foundations for more complex algorithms and systems, making them essential tools in any programmer's toolkit.

6.7 FURTHER READINGS

1. Books:

- **"Data Structures and Algorithms in Python" by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser:** This book covers fundamental data structures, including stacks, queues, and deques, with a focus on implementation in Python.
- **"Python Data Structures and Algorithms" by Benjamin Baka:** This book offers a practical approach to implementing various data structures in Python, including detailed sections on stacks and queues.

2. Online Courses:

- **Coursera - "Data Structures and Algorithms Specialization":** This course series includes modules specifically focused on stacks and queues, providing both theoretical knowledge and practical coding exercises.
- **edX - "Introduction to Computer Science using Python":** This course covers basic data structures including stacks and queues with hands-on programming assignments.

3. Documentation and Tutorials:

- **Python Official Documentation:** The official Python documentation provides insights into the collections module where deques are implemented. It includes usage examples and performance considerations.
- **GeeksforGeeks - Data Structures:** This website offers tutorials on various data structures including stacks, queues, and deques with examples in Python.
- **Real Python - "Understanding Deque in Python":** A detailed article that explains the deque data structure with examples of its usage in real-world applications.

UNIT 7 LINKED LISTS

Structure

- 7.0 Introduction
- 7.1 Objectives
- 7.2 Singly Linked Lists
- 7.3 Circularly Linked Lists
- 7.4 Doubly Linked Lists
- 7.5 The Positional List ADT
- 7.6 Sorting a Positional List
- 7.7 Case Study: Maintaining Access Frequencies
- 7.8 Link-Based vs. Array-Based Sequences
- 7.9 Summary
- 7.10 Further Readings

7.0 Introduction

Unlike arrays that store elements in contiguous memory locations, linked lists provide a dynamic way to organize data using nodes. Each node contains both data and a next node reference in the sequence, creating a chain-like structure that can grow or shrink dynamically during program execution.

This unit explores various types of linked lists, including singly linked lists, doubly linked lists, circular linked lists, and the positional list ADT.

7.1 Objectives

After completing this unit, you will be able to:

- Explain the fundamental concepts and components of linked list data structures, including nodes, references, and dynamic memory allocation
- Differentiate between various types of linked lists (singly, doubly, circular) and understand their specific use cases
- Implement list operations including insertion, traversal and deletion.
- Analyze the time and space complexity of various linked list operations
- Compare linked lists with arrays and understand when to choose one over the other
- Design and implement the positional list ADT using a doubly linked list structure
- Debug common linked list problems such as cycle detection and reference management
- Apply linked list concepts to solve real-world programming problems

7.2 Singly Linked List

Singly Linked Lists provides efficient data management and manipulation. Unlike arrays, which have a fixed size and require reshuffling during insertion or deletion, singly linked lists provide dynamic memory allocation, enabling easier modifications. This chapter details the structure, implementation, and applications of singly linked lists in Python.

A Singly Linked List consists of a sequence of nodes, where each node contains two components:

- Data: Node's value
- Next Pointer: Next node's pointer in the sequence.

STRUCTURE OF A NODE

In Python, a node can be represented using a class:

```
class Node:
    def __init__(self, data):
self.data = data # Store the data
self.next = None # Initialize the next pointer to None
'''
```

IMPLEMENTATION OF SINGLY LINKED LISTS

The singly linked list itself can be implemented as follows:

```
class SinglyLinkedList:
    def __init__(self):
self.head = None # Initialize

    def insert_at_end(self, data):
new_node = Node(data)
    if not self.head: # empty list
self.head = new_node
        return
last_node = self.head
        while last_node.next: # last node traversal
last_node = last_node.next
last_node.next = new_node # new node link

    def display(self):
current = self.head
        while current: # Traverse and print each node's data
print(current.data, end=" -> ")
            current = current.next
        print("None") # Indicate the end of the list
'''
```

KEY OPERATIONS ON SINGLY LINKED LISTS

INSERTION

Insertion can occur at various positions within the list:

- At the beginning
- At the end
- At a specific position

Example: The following Python code has three insert operations – insert at the beginning, insert at the end and insert after a node.

```
class SinglyLinkedList:
    def insert_at_beginning(self, data):
new_node = Node(data)
new_node.next = self.head
```

```
self.head = new_node
```

```
def insert_at_end(self, data):
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
    return
```

```
current = self.head
while current.next:
    current = current.next
current.next = new_node
```

```
def insert_after_node(self, prev_node, data):
    if prev_node is None:
        print("Previous node cannot be None")
    return
```

```
new_node = Node(data)
new_node.next = prev_node.next
prev_node.next = new_node
```

DELETION

Deletion operations can also target specific nodes:

- Delete by value
- Delete at a specific position

Example: Deleting by Value

```
def delete_by_value(self, value):
    current = self.head

    if current and current.data == value: # Check if head needs to be deleted
        self.head = current.next # Update head to next node
    return
```

```
prev = None
while current and current.data != value: # Traverse until value is found
    prev = current
    current = current.next
```

```
if not current: # Value not found in list
    return
```

```
prev.next = current.next # Bypass the deleted node
'''
```

Another example of the delete operation is given below. The code supports deletion from the beginning, from the end, and from a specific position.

```
class SinglyLinkedList:
    def delete_from_beginning(self):
```

```

    if self.head is None:
        return None

    temp = self.head
    self.head = self.head.next
    return temp.data

    def delete_from_end(self):
        if self.head is None:
            return None

        if self.head.next is None:
            temp = self.head
            self.head = None
            return temp.data

        current = self.head
        while current.next.next:
            current = current.next

        temp = current.next
        current.next = None
        return temp.data

    def delete_node(self, key):
        if self.head is None:
            return

        if self.head.data == key:
            self.head = self.head.next
            return

        current = self.head
        while current.next and current.next.data != key:
            current = current.next

        if current.next:
            current.next = current.next.next

```

PRACTICAL USE CASES OF SINGLY LINKED LISTS

- Dynamic Memory Allocation: Singly linked lists are ideal for applications where memory utilization needs to be efficient and dynamic.
- Implementing Stacks and Queues: They can serve as foundational structures for more complex data types such as stacks (LIFO) and queues (FIFO).
- Real-time Applications: In scenarios like browser history or undo functionality in applications, singly linked lists allow for quick insertions and deletions.

BEST PRACTICES FOR USING SINGLY LINKED LISTS

- Memory Management: Always ensure that nodes are properly deallocated when they are no longer needed to avoid memory leaks.

- Error Handling: Implement checks for edge cases such as deletion from an empty list or accessing out-of-bounds indices.
- Maintainability: Keep your code modular by separating logic into distinct methods for insertion, deletion, and traversal.
- Handle edge cases: Check for the edge cases such as empty list, single node list, and operations at the beginning or at the end.

Head reference: Always maintain head reference and update the head reference during insert/delete operations.

Time complexity of various operations

The table provides the time complexity of various operations on singly linked list

Operation	Time complexity	Comments
Insert at beginning	$O(1)$	Constant time operation
Insert at end	$O(n)$	Need to traverse to end
Delete from beginning	$O(1)$	Constant time operation
Delete from end	$O(n)$	Need to traverse to end
Search	$O(n)$	May need to traverse entire list
Access by index	$O(n)$	Need to traverse to index

Singly linked lists are versatile data structures that offer significant advantages over traditional arrays in terms of flexibility and efficiency. By understanding their implementation and key operations in Python, you can effectively utilize them in various programming scenarios. As you continue your programming journey, consider how singly linked lists can enhance your applications through dynamic data management capabilities.

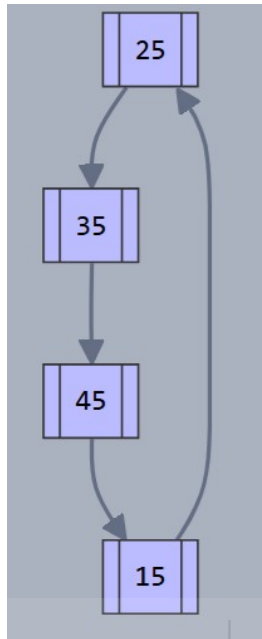
7.3 Circularly Linked List

In the study of data structures, Circularly Linked Lists present a unique variation of linked lists where the last node points back to the first node, forming a circular structure. This configuration allows for more efficient traversal and manipulation of data, particularly in scenarios requiring continuous looping through the list. This chapter will explore the intricacies of circularly linked lists, their implementation in Python, and their practical applications.

A Circularly Linked List is a collection of nodes where each node contains two components:

- Data: The value stored in the node.
- Next Pointer: A reference to the next node in the sequence, with the last node pointing back to the first node.

A visual representation of circularly linked list is given below –



IMPLEMENTATION OF CIRCULARLY LINKED LISTS

The circularly linked list can be implemented as follows:

```

class CircularLinkedList:
    def __init__(self):
        self.head = None # Initialize the head of the list

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head: # If the list is empty
            self.head = new_node
            new_node.next = self.head # Point to itself
            return
        last_node = self.head
        while last_node.next != self.head: # Traverse to find the last node
            last_node = last_node.next
        last_node.next = new_node # Link new node at end
        new_node.next = self.head # Point new node back to head

    def display(self):
        if not self.head:
            print("List is empty.")
            return
        current = self.head
        while True: # Traverse until we loop back to head
            print(current.data, end=" -> ")
            current = current.next
            if current == self.head:
                break
        print("(head)")
  
```

KEY OPERATIONS ON CIRCULARLY LINKED LISTS**INSERTION**

Insertion can occur at various positions within a circularly linked list:

- At the beginning
- At the end
- At a specific position

An example code for insert at various positions is given below -

```
class CircularLinkedList:
    def insert_at_beginning(self, data):
        new_node = Node(data)
        if self.is_empty():
            new_node.next = new_node # Points to itself
            self.head = new_node
        else:
            # Check last node
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.is_empty():
            new_node.next = new_node
            self.head = new_node
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head

    def insert_after(self, prev_data, data):
        if self.is_empty():
            return

        temp = self.head
        while True:
            if temp.data == prev_data:
                new_node = Node(data)
                new_node.next = temp.next
                temp.next = new_node
                break
            temp = temp.next
            if temp == self.head:
                break
```

ignou
THE PEOPLE'S
UNIVERSITY

DELETION

Deletion operations can target specific nodes by value or position.

Example: Deleting by Value

```
def delete_by_value(self, value):
    if not self.head:
        return "List is empty."

    current = self.head

    if current.data == value: # Check if head needs to be deleted
        if current.next == self.head: # Only one node in list
            self.head = None
            return "Node deleted."

    while current.next != self.head: # Find last node
        current = current.next

    current.next = self.head.next # Bypass head node and update head
    self.head = self.head.next
    return "Node deleted."

prev = None
while current.data != value and current.next != self.head:
    prev = current
    current = current.next

    if current.data == value:
        prev.next = current.next
        return "Node deleted."

    return "Value not found."
'''
```

PRACTICAL USE CASES OF CIRCULARLY LINKED LISTS

- Round Robin Scheduling: In operating systems, circularly linked lists processes scheduling where each process is given an equal share of CPU time in a cyclic manner.
- Multiplayer Games: They can manage players in a round-robin fashion where they can play in turns.
- Music Playlists: Circularly linked lists can create playlists that loop continuously.

BEST PRACTICES FOR USING CIRCULARLY LINKED LISTS

- Memory Management: Deallocate nodes when not required to prevent memory leaks.

- Error Handling: Implement checks for edge cases such as deleting from an empty list or accessing out-of-bounds indices.
- Maintainability: Keep your code modular by separating logic into distinct methods for insertion, deletion, and traversal.

Circularly linked lists provide an efficient way to manage data that requires continuous looping through elements. By understanding their implementation and key operations in Python, you can effectively utilize them in various programming scenarios. As you advance your knowledge of data structures, consider how circularly linked lists can enhance your applications through their dynamic and flexible nature.

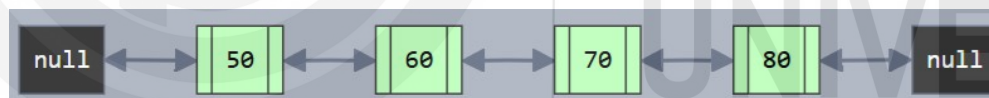
7.4 Doubly linked lists

In the domain of data structures, Doubly Linked Lists offer a more versatile alternative to singly linked lists by allowing traversal in both directions—forward and backward. This capability enhances the efficiency of certain operations, such as deletion and insertion, making doubly linked lists particularly useful in applications requiring bidirectional access to data. This chapter will provide an in-depth exploration of doubly linked lists, their implementation in Python, and their practical applications.

In a Doubly Linked List each node contains three components:

- Data: The value stored in the node.
- Next Pointer: next node's reference in the sequence.
- Previous Pointer: Previous node's reference in the sequence.

A sample doubly linked list is given below -



STRUCTURE OF A NODE

In Python, a node can be represented using a class:

```

class Node:
    def __init__(self, data):
self.data = data # Store the data
self.next = None # Initialize the next pointer to None
self.prev = None # Initialize the previous pointer to None
'''
  
```

IMPLEMENTATION OF DOUBLY LINKED LISTS

The doubly linked list can be implemented as follows:

```

class DoublyLinkedList:
    def __init__(self):
self.head = None # Initialize

    def insert_at_end(self, data):
  
```

Linked List

```
new_node = Node(data)
    if not self.head: # empty list
self.head = new_node
    return
last_node = self.head
    while last_node.next: # Go to last node
last_node = last_node.next
last_node.next = new_node # Link last node to new node
new_node.prev = last_node # Link new node back to last node

def display(self):
    current = self.head
    while current: # Traverse and print each node's data forward
print(current.data, end=" <-> ")
        current = current.next
    print("None") # Indicate the list's end

def display_reverse(self):
    current = self.head
    if not current:
        return "List is empty."

    while current.next: # Traverse to the last node
        current = current.next

    while current: # Traverse and print each node's data backward
print(current.data, end=" <-> ")
        current = current.prev
    print("None") # Indicate the end of the list
...
```

KEY OPERATIONS ON DOUBLY LINKED LISTS

INSERTION

Insertion can occur at various positions within a doubly linked list:

- At the beginning
- At the end
- At a specific position

An example code for insert at various positions is given below -

```
class DoublyLinkedList:
    def insert_at_beginning(self, data):
new_node = Node(data)
        if self.head is None:
self.head = new_node
self.tail = new_node
        else:
new_node.next = self.head
self.head.prev = new_node
self.head = new_node
```

```
def insert_at_end(self, data):
```

```

new_node = Node(data)
    if self.tail is None:
self.head = new_node
self.tail = new_node
    else:
self.tail.next = new_node
new_node.prev = self.tail
self.tail = new_node

def insert_after_node(self, node, data):
    if node is None:
        return

```

```

new_node = Node(data)
new_node.next = node.next
new_node.prev = node

```

```

    if node.next:
node.next.prev = new_node
node.next = new_node

```

```

    if node == self.tail:
self.tail = new_node

```

DELETION

Deletion operations can target specific nodes by value or position.

```

class DoublyLinkedList:
    def delete_from_beginning(self):
        if self.head is None:
            return None

```

```

        temp = self.head
self.head = self.head.next
        if self.head:
self.head.prev = None
        else:
self.tail = None
        return temp.data

```

```

    def delete_from_end(self):
        if self.tail is None:
            return None

```

```

        temp = self.tail
self.tail = self.tail.prev
        if self.tail:
self.tail.next = None
        else:
self.head = None
        return temp.data

```

```

def delete_node(self, node):

```

```

    if node is None:
        return

    if node.prev:
        node.prev.next = node.next
    else:
        self.head = node.next

    if node.next:
        node.next.prev = node.prev
    else:
        self.tail = node.prev

```

PRACTICAL USE CASES OF DOUBLY LINKED LISTS

- Browser History Management: Doubly linked lists are ideal for implementing features like back and forward navigation in web browsers since they allow traversal in both directions.
- Music Playlists: They can manage playlists where users may want to move both forwards and backwards through songs.
- Undo/Redo Functionality: Applications that require undo and redo capabilities can utilize doubly linked lists effectively.

BEST PRACTICES FOR USING DOUBLY LINKED LISTS

- Memory Management: Ensure that nodes are properly deallocated when they are no longer needed to prevent memory leaks.
- Error Handling: Implement checks for edge cases such as deleting from an empty list or accessing out-of-bounds indices.
- Maintainability: Keep your code modular by separating logic into distinct methods for insertion, deletion, and traversal.

Time complexity of various operations

The table provides the time complexity of various operations on doubly linked list

Operation	Time complexity	Comments
Insert at beginning	O(1)	With head pointer
Insert at end	O(1)	With tail pointer
Delete from beginning	O(1)	With head pointer
Delete from end	O(1)	With tail pointer
Search	O(n)	Sequential search
Insert after node	O(1)	With node reference
Delete node	O(1)	With node reference

Doubly linked lists provide enhanced flexibility for managing data that requires bidirectional access. By understanding their implementation and key operations in Python, you can effectively utilize them in various programming scenarios. As

you continue your exploration of data structures, consider how doubly linked lists can improve your applications through their dynamic nature and efficient manipulation capabilities.

7.5 The positional ADT

In the study of data structures, the Positional List Abstract Data Type (ADT) provides a flexible way to manage a sequence of elements where each element can be accessed based on its position. Unlike traditional lists, positional lists allow for efficient insertion and deletion operations at arbitrary positions, making them suitable for applications that require frequent modifications. This chapter will explore the concept of positional lists, their implementation in Python, and practical use cases.

A Positional List is an abstract data type that allows elements to be stored in a sequence while providing direct access to each element by its position. This structure typically supports operations such as insertion, deletion, and retrieval at specified positions.

A positional list is depicted in the below diagram -

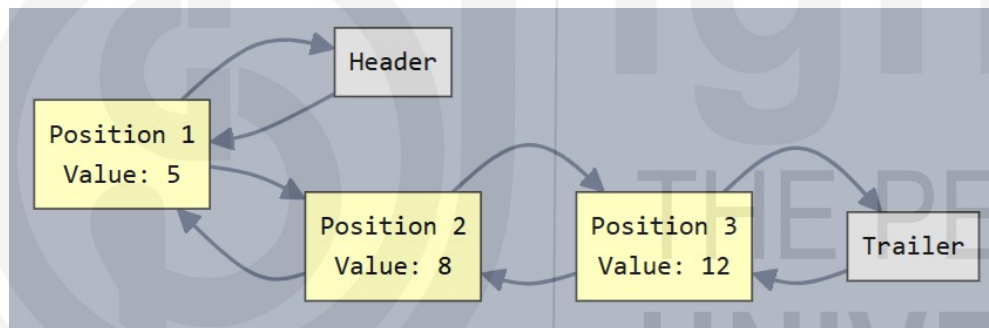


Figure 7.1 : Positional List

STRUCTURE OF A NODE

In Python, a node in a positional list can be represented using a class that contains references to both the next and previous nodes:

```
class Node:
    def __init__(self, data):
self.data = data # Store the data
self.next = None # Pointer to the next node
self.prev = None # Pointer to the previous node
'''
```

IMPLEMENTATION OF POSITIONAL LISTS

The positional list can be implemented using a doubly linked list structure where each node maintains pointers to its neighboring nodes:

```
class PositionalList:
    def __init__(self):
```

Linked List

```
self.head = None # Initialize the head of the list
self.tail = None # Initialize the tail of the list
self.size = 0    # Track the number of elements

def insert_at_position(self, data, position):
    new_node = Node(data)
    if position < 0 or position > self.size:
        raise IndexError("Position out of bounds.")

    if position == 0: # Insert at the beginning
        if not self.head: # If list is empty
            self.head = new_node
            self.tail = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
    elif position == self.size: # Insert at the end
        new_node.prev = self.tail
        if self.tail:
            self.tail.next = new_node
            self.tail = new_node
        else: # Insert in the middle
            current = self.head
            for _ in range(position):
                current = current.next

            new_node.prev = current.prev
            new_node.next = current

            if current.prev:
                current.prev.next = new_node

            current.prev = new_node

    self.size += 1

def delete_at_position(self, position):
    if position < 0 or position >= self.size:
        raise IndexError("Position out of bounds.")

    if position == 0: # Delete from the beginning
        if self.head == self.tail: # Only one element in the list
            self.head = None
            self.tail = None
        else:
            self.head = self.head.next
            if self.head:
                self.head.prev = None

    elif position == self.size - 1: # Delete from the end
        if self.tail:
            self.tail = self.tail.prev
            if self.tail:
```

```

self.tail.next = None

    else: # Delete from the middle
        current = self.head
        for _ in range(position):
            current = current.next

        if current.prev:
            current.prev.next = current.next

        if current.next:
            current.next.prev = current.prev

self.size -= 1

def display(self):
    current = self.head
    while current:
        print(current.data, end=" <-> ")
        current = current.next
    print("None")
...

```

KEY OPERATIONS ON POSITIONAL LISTS

INSERTION AND DELETION

Insertion and deletion operations are required to maintain the integrity of a positional list. The provided methods allow for insertion at any specified position and deletion from any specified position.

Example: Inserting Elements

```

pl = PositionalList()
pl.insert_at_position(10, 0) # Insert at beginning
pl.insert_at_position(20, 1) # Insert at end
pl.insert_at_position(15, 1) # Insert in middle (at index 1)
pl.display()                # Output: 10 <-> 15 <-> 20 <-> None
...

```

Example: Deleting Elements

```

pl.delete_at_position(1)    # Delete element at index 1 (15)
pl.display()                # Output: 10 <-> 20 <-> None
...

```

PRACTICAL USE CASES OF POSITIONAL LISTS

- Text Editor Functionality: Positional lists can be used to implement features such as undo/redo functionality where text can be inserted or deleted at specific positions.

- Dynamic Playlist Management: In music applications, users can add or remove songs from specific positions in their playlists.

- Scheduling Tasks: In task management applications, tasks can be dynamically added or removed based on user input.

BEST PRACTICES FOR USING POSITIONAL LISTS

- Memory Management: Deallocate the nodes when they are no longer required to prevent memory leaks.
- Error Handling: Implement checks for edge cases such as inserting or deleting from an empty list or accessing out-of-bounds indices.
- Maintainability: Keep your code modular by separating logic into distinct methods for insertion, deletion, and traversal.

The Positional List ADT provides a powerful way to manage sequences of elements with efficient insertion and deletion capabilities at arbitrary positions. By understanding its implementation and key operations in Python, you can effectively utilize this data structure in various programming scenarios. As you continue your exploration of data structures, consider how positional lists can enhance your applications through their dynamic nature and flexibility.

7.6 Sorting a positional list

Sorting is a fundamental operation in computer science that arranges the elements of a data structure in a specific order, typically ascending or descending. When dealing with a Positional List, sorting becomes essential for efficiently managing and retrieving data.

SORTING ALGORITHMS OVERVIEW

Several sorting algorithms can be utilized to sort elements within a positional list. The choice of algorithm may depend on factors such as the list size, the nature of the data, and performance requirements. Common sorting algorithms include:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Quick Sort

For this chapter, we will focus on implementing Insertion Sort and Merge Sort, as they are particularly well-suited for linked structures like positional lists.

INSERTION SORT FOR POSITIONAL LISTS

Insertion Sort is a good fit for sorting small datasets. Each element is compared one at a time with each new element with those already sorted to build a sorted array.

IMPLEMENTATION OF INSERTION SORT

```

def insertion_sort(positional_list):
    if positional_list.head is None:
        return # List is empty

    sorted_list = PositionalList()
    current = positional_list.head

    while current:
        # Insert current node into sorted_list at the correct position
        insert_sorted(sorted_list, current.data)
        current = current.next

    return sorted_list

def insert_sorted(sorted_list, data):
    new_node = Node(data)
    if not sorted_list.head or sorted_list.head.data >= data: # Insert at head
        new_node.next = sorted_list.head
        if sorted_list.head:
            sorted_list.head.prev = new_node
        sorted_list.head = new_node
    else:
        current = sorted_list.head
        while current.next and current.next.data < data: # Find position to
            insert
            current = current.next
        new_node.next = current.next
        if current.next: # If not inserting at end
            current.next.prev = new_node
        current.next = new_node
        new_node.prev = current
    ``

```

5. MERGE SORT FOR POSITIONAL LISTS

Merge Sort is a divide-and-conquer algorithm that divides the list into halves, sorts each half, and then merges them back together.

IMPLEMENTATION OF MERGE SORT

```

def merge_sort(positional_list):
    if positional_list.size <= 1:
        return positional_list

    mid = positional_list.size // 2
    left_half = PositionalList()
    right_half = PositionalList()

    # Split the original list into two halves
    current = positional_list.head

```

```

        for _ in range(mid):
            left_half.insert_at_end(current.data)
            current = current.next

        while current: # Remaining elements go to right_half
            right_half.insert_at_end(current.data)
            current = current.next

        # Recursively sort both halves and merge them back together
        return merge(merge_sort(left_half), merge_sort(right_half))

def merge(left, right):
    merged = PositionalList()
    left_current = left.head
    right_current = right.head

    while left_current and right_current:
        if left_current.data <= right_current.data:
            merged.insert_at_end(left_current.data)
            left_current = left_current.next
        else:
            merged.insert_at_end(right_current.data)
            right_current = right_current.next

    while left_current: # Append remaining elements from left half
        merged.insert_at_end(left_current.data)
        left_current = left_current.next

    while right_current: # Append remaining elements from right half
        merged.insert_at_end(right_current.data)
        right_current = right_current.next

    return merged
'''

```

PRACTICAL USE CASES FOR SORTING POSITIONAL LISTS

- Data Organization: Sorting is essential in applications where data needs to be presented in an organized manner, such as contact lists or student records.
- Search Optimization: Sorted lists enable faster search operations using binary search techniques.
- Task Management: In task scheduling applications, tasks can be sorted by priority or deadlines to optimize workflow.

BEST PRACTICES FOR SORTING POSITIONAL LISTS

- Choose the Right Algorithm: Depending on the size and nature of your data, select an appropriate sorting algorithm that balances efficiency and simplicity.

- Consider Stability: If maintaining the relative order of equal elements is important, choose a stable sorting algorithm like Merge Sort.
- Optimize Memory Usage: Be mindful of memory consumption when implementing sorting algorithms, especially with large datasets.

Sorting a positional list is required for data retrieval and data management.. By understanding and implementing various sorting algorithms in Python, you can enhance the functionality of your applications significantly. As you continue to explore data structures and algorithms, consider how sorting can improve user experience and application performance through organized data presentation.

7.7 Case study: Maintaining Access Frequencies in positional list

In many applications, it is essential to maintain not only the data itself but also metadata about how frequently each element is accessed. This case study details how to implement access frequency tracking in a Positional List. By doing so, we can optimize operations based on usage patterns, which is particularly useful in scenarios like caching, user activity tracking, and resource management.

BACKGROUND

Access frequency refers to how often an element in a data structure is accessed or modified. We can use the access frequency to improve the performance of the data retrieval by caching the frequently accessed elements.

IMPLEMENTATION OF ACCESS FREQUENCY IN A POSITIONAL LIST

To implement a positional list that maintains access frequencies, we can extend the basic structure of a positional list by adding an additional attribute to each node that tracks its access count.

NODE STRUCTURE WITH ACCESS FREQUENCY

We will modify our existing 'Node' class to include an 'access_count' attribute.

```
class Node:
    def __init__(self, data):
        self.data = data      # Store the data
        self.next = None     # Pointer to the next node
        self.prev = None     # Pointer to the previous node
        self.access_count = 0 # Initialize access count to zero
    ``
```

MODIFIED POSITIONAL LIST CLASS

The 'PositionalList' class will be updated to include methods for accessing elements and incrementing their access counts.

```
class PositionalList:
    def __init__(self):
        self.head = None    # Initialize
        self.tail = None    # Initialize the list's tail
```

```

self.size = 0          # Track elements' number

def insert_at_position(self, data, position):
    new_node = Node(data)
    if position < 0 or position > self.size:
        raise IndexError("Position out of bounds.")

    if position == 0: # Insert at the beginning
        if not self.head: # If list is empty
            self.head = new_node
            self.tail = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
    elif position == self.size: # Insert at the end
        new_node.prev = self.tail
        if self.tail:
            self.tail.next = new_node
            self.tail = new_node
        else: # Insert in the middle
            current = self.head
            for _ in range(position):
                current = current.next

        new_node.prev = current.prev
        new_node.next = current

        if current.prev:
            current.prev.next = new_node
        current.prev = new_node

    self.size += 1

def access_element(self, position):
    if position < 0 or position >= self.size:
        raise IndexError("Position out of bounds.")

    current = self.head
    for _ in range(position):
        current = current.next

    current.access_count += 1 # Increment access count on retrieval
    return current.data

def display_access_frequencies(self):
    current = self.head
    while current:
        print(f'Data: {current.data}, Access Count: {current.access_count}')
        current = current.next
    ...

```

PRACTICAL USE CASES FOR MAINTAINING ACCESS FREQUENCIES

- Caching Mechanisms: In systems where data retrieval speed is critical (e.g., web caching), maintaining access frequencies can help to identify the items that should remain in cache and which should be evicted.
- User Behavior Analysis: Applications that track user interactions can be improved by understanding which features or items are accessed most frequently, allowing for better UI/UX design.
- Resource Management: In resource allocation scenarios (e.g., memory management), maintaining access frequencies helps prioritize which resources to keep active based on usage patterns.

BEST PRACTICES FOR ACCESS FREQUENCIES

- Efficient Updates: Ensure that updating access counts does not significantly impact performance, especially in high-frequency access scenarios.
- Memory Management: Regularly clean up nodes that are no longer needed to prevent memory leaks and maintain performance.
- Data Structure Choice: Choose the appropriate underlying data structure based on expected usage patterns and performance requirements.

By implementing access frequency feature in positional lists, applications can become more efficient and responsive to user behavior and system demands. As you explore further into data structures and algorithms, consider how tracking metadata like access frequencies can optimize your applications' performance and user experience.

7.8 Link-Based vs. Array-Based Sequences

Sequences store and manage collections of elements. Two primary implementations of sequences are array-based sequences and link-based sequences (or linked lists). This section will explore these two approaches, examining their strengths and weaknesses in various scenarios.

ARRAY-BASED SEQUENCES

Array-based sequences utilize a contiguous block of memory to store elements. Given below are advantages and disadvantages:

Advantages

- Direct Access: Elements can be accessed in constant time $O(1)$ using their index, making retrieval operations very efficient.
- Cache Performance: Since elements are stored contiguously, array-based sequences benefit from better cache locality, leading to faster access times.

Disadvantages

- Fixed Size: The size of an array must be defined at the time of creation. Resizing an array requires allocating a new larger array and copying elements, which can be time-consuming.
- Inefficient Insertions/Deletions: Inserting or deleting elements (especially in the middle or at the beginning) requires shifting subsequent elements, resulting in $O(n)$ time complexity for these operations.

LINK-BASED SEQUENCES

Link-based sequences, or linked lists, have nodes that point to next and previous nodes. This structure provides a different set of advantages and disadvantages:

Advantages

- Dynamic Size: Linked lists can grow and shrink dynamically as needed without the need for resizing or reallocating memory.
- Efficient Insertions/Deletions: Addition or deletion of nodes can be done in constant time $O(1)$ if the position is known (e.g., at the beginning or end), making these operations much more efficient compared to arrays.

Disadvantages

- No Direct Access: Accessing an element requires traversal from the list's head, leading to $O(n)$ time complexity for retrieval operations.
- Memory Overhead: Each node needs extra memory for storing pointers, which can lead to increased memory usage compared to arrays, especially for small data types.

COMPARATIVE ANALYSIS

The table compares the array-based sequences and link based sequences –

Feature	Array-Based Sequences	Link based sequences
Memory Allocation	Contiguous block	Dynamic allocation (nodes)
Access Time	$O(1)$ (direct access)	$O(n)$ (sequential access)
Insertion/Deletion time	$O(n)$ (requires shifting)	$O(1)$ (if position is known)
Size	Fixed size	Dynamic size
Memory Overhead	Minimal	Higher due to pointers
Cache Performance	Better due to locality	Poorer due to scattered memory

PRACTICAL USE CASES

When to Use Array-Based Sequences

- When the number of elements is known in advance and does not change frequently.
- When fast access times are critical, such as in applications requiring frequent read operations.

When to Use Link-Based Sequences

- When frequent insertions and deletions are expected.
- In applications where the size of the dataset can vary significantly over time.

Both array-based and link-based sequences have their unique advantages and trade-offs. Understanding these differences is crucial for selecting the appropriate data structure based on specific application requirements. Using memory usage, access patterns, we can optimize performance and resource utilization in their applications.

Check Your Progress

1. **What is the main difference between a singly linked list and a doubly linked list?** a) A singly linked list needs more memory than a doubly linked list b) A doubly linked list can only traverse forward, while a singly linked list traverses both directions c) A singly linked list node contains one reference (next), while a doubly linked list node has two references (next and previous) d) A doubly linked list can only store numeric data, while a singly linked list can store any data type Correct Answer: c)
2. **In a circular linked list with 5 nodes, what does the 'next' pointer of the last node point to?** a) NULL b) The first node of the list c) The second to last node d) Undefined memory location Correct Answer: b)
3. **What is the time complexity of inserting a new node at a known position in a doubly linked list?** a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n^2)$ Correct Answer: c) $O(1)$
4. **Which algorithm is commonly used to detect a cycle in a linked list?** a) Binary Search Algorithm b) Floyd's Cycle-Finding Algorithm (Tortoise and Hare) c) Quicksort Algorithm d) Depth-First Search Correct Answer: b) Floyd's Cycle-Finding Algorithm (Tortoise and Hare)
5. **What is the primary advantage of using a positional list ADT?** a) It consumes less memory than other list implementations b) It provides faster access to elements c) It provides a higher level of abstraction and encapsulation d) It can only store unique elements Correct Answer: c) It provides a higher level of abstraction and encapsulation
6. **When deleting a node from a doubly linked list in a language without garbage collection, what is essential?** a) Only update the next references b) Only update the previous references c) Update both previous and next references before freeing memory d) Immediately allocate the freed memory to a new node Correct Answer: c) Update both previous and next references before freeing memory
7. **What is the time complexity of removing the last node in a doubly linked list?** a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$ Correct Answer: a) $O(1)$
8. **Which scenario would be best suited for a circular linked list?** a) Implementing a stack b) Managing a round-robin scheduling system c) Storing a binary search tree d) Implementing a hash table Correct Answer: b) Managing a round-robin scheduling system

7.9 Summary

Linked lists serve as a cornerstone of data structure implementation, offering flexibility and efficiency for specific operations that arrays cannot match. It needs more memory due to the storage of references and potentially suffer from lack of

random access, their ability to grow dynamically and efficiently handle insertions and deletions at known positions makes them invaluable in many applications.

Through this unit, we've seen how different variations of linked lists can be adapted to solve specific problems. Singly linked lists offer memory efficiency, doubly linked lists provide bidirectional traversal, circular lists enable cyclic operations, and positional lists abstract the implementation details behind a clean interface.

7.10 Further Readings

Books

- "Data Structures and Algorithms in Python" by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser
- "Python Data Structures and Algorithms" by Benjamin Baka

Online Resources

- [GeeksforGeeks: Linked List Data Structure](#)



UNIT 8 TREES

Structure

- 8.0 Introduction
- 8.1 Objectives
- 8.2 General Trees
- 8.3 Binary Trees
- 8.4 Implementing Trees
- 8.5 Tree Traversal Algorithms
- 8.6 Case Study: An Expression Tree
- 8.7 Summary
- 8.8 Further Readings

8.0 Introduction

Trees arrange elements in hierarchical structure. They provide an efficient way to represent and manage hierarchical relationships between data elements. Trees find extensive applications in file systems, database indexing, syntax parsing, and organizational hierarchies.

8.1 Objectives

After completing this unit, you will be able to:

- Understand the fundamental concepts and terminology of tree data structures
- Differentiate between general trees and binary trees
- Implement basic tree operations including insertion, deletion, and traversal
- Apply different tree traversal algorithms (pre-order, in-order, post-order, level-order)
- Create and evaluate expression trees
- Analyze the time complexity of various tree operations

8.2 General Trees

We use Trees to manage and identify hierarchical relationship between elements. Data that has parent-child relationships such as manager-employee, teacher-student, organization-sub organization can be stored in tree data structure.

In this section we closely look at the definition, structure and use cases for tree data structure.

GENERAL TREES: DEFINITION AND STRUCTURE

Nodes are main elements of tree data structure. Each node has zero or multiple child nodes. This flexibility allows general trees to represent various types of hierarchical data, such as file systems or organizational structures.

TERMINOLOGY

- Node: The fundamental part of a tree that contains data and links to its children.
- Root: The topmost node of the tree.
- Leaf: A node without any children.
- Subtree: A tree formed by a node and its descendants.

TRAVERSING GENERAL TREES

Traversal refers to visiting all nodes of the tree in a specific order. Common traversal methods include pre-order, post-order, and level-order traversals.

PRE-ORDER TRAVERSAL

Root node is visited first after which we visit the child nodes in pre-order traversal.

```
def pre_order_traversal(node):
    if node:
        print(node.data)
        for child in node.children:
            pre_order_traversal(child)
```

```
# Example usage
pre_order_traversal(root)
'''
```

POST-ORDER TRAVERSAL

All the child nodes are visited before the root node in post order traversal.

```
def post_order_traversal(node):
    if node:
        for child in node.children:
            post_order_traversal(child)
        print(node.data)
```

```
# Example usage
post_order_traversal(root)
'''
```

Sample Python code for Depth first travel and breadth first traversals are given below

```
def depth_first_traversal(node, level=0):
    """Print tree nodes in depth-first order"""
    print(" " * level + node.data)
    for child in node.children:
        depth_first_traversal(child, level + 1)
```

```
def breadth_first_traversal(root):
    """Print tree nodes in breadth-first order"""
    if not root:
        return
    queue = [root]
    while queue:
        current = queue.pop(0)
        print(current.data)
        queue.extend(current.children)
```

Node Operations

Common operations include adding child nodes, removing nodes, and searching for specific nodes:

```
class TreeNode:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.children = []
```

```
        self.parent = None
```

```
    def add_child(self, child):
```

```
        child.parent = self
```

```
        self.children.append(child)
```

```
    def remove_child(self, child):
```

```
        if child in self.children:
```

```
            child.parent = None
```

```
            self.children.remove(child)
```

PRACTICAL USE CASES OF GENERAL TREES

Given below are few real-world use cases for trees -

File System Organization

- Directories containing files and subdirectories
- Each directory can have multiple children

Organization Charts

- Representing company hierarchies
- Department structures

XML/HTML DOM

- Representing document structure with hierarchical structure

Menu Systems

- Application menu hierarchies
- Game skill trees

BEST PRACTICES

1. When implementing general trees, consider these best practices:
2. Memory Management
 - Implement proper cleanup methods
 - Handle circular references carefully
3. Performance Optimization
 - Choose appropriate traversal methods based on use case
 - Consider caching frequently accessed nodes
4. Data Validation
 - Validate node relationships before modifications
 - Implement error checking for operations
5. Design Considerations
 - Keep the tree balanced when possible
 - Use appropriate node structures for the specific use case

Case study – Simple File system structure

Here's an example implementing a simple file system structure:

```

class FileSystemNode(TreeNode):
    def __init__(self, name, is_directory=True):
        super().__init__(name)
        self.is_directory = is_directory

    def display(self, level=0):
        prefix = " " * level
        file_type = "dir" if self.is_directory else "file"
        print(f'{prefix} {self.data} [{file_type}]')
        for child in self.children:
            child.display(level + 1)

# Usage example
root = FileSystemNode("root")
documents = FileSystemNode("Documents")
photos = FileSystemNode("Photos")
report = FileSystemNode("report.doc", False)
vacation = FileSystemNode("vacation.jpg", False)

root.add_child(documents)
root.add_child(photos)
documents.add_child(report)
photos.add_child(vacation)

root.display()

```

General trees are flexible in representing hierarchical relationships. By understanding their implementation and traversal methods in Python, you can effectively utilize them in various applications ranging from file systems to organizational charts. As you continue your journey in programming, mastering general trees will enhance your ability to solve complex problems efficiently.

8.3 Binary Trees

In Binary tree data structure each node has left child node and right child node. This chapter will provide an in-depth exploration of binary trees, including their definition, implementation in Python, traversal techniques, practical applications, and best practices.

BINARY TREES: DEFINITION AND STRUCTURE

A node of binary tree consist of -

- A data element.
- A reference to the left child (which is also a binary tree).
- A reference to the right child (which is also a binary tree).

A sample Binary tree is depicted in the following diagram

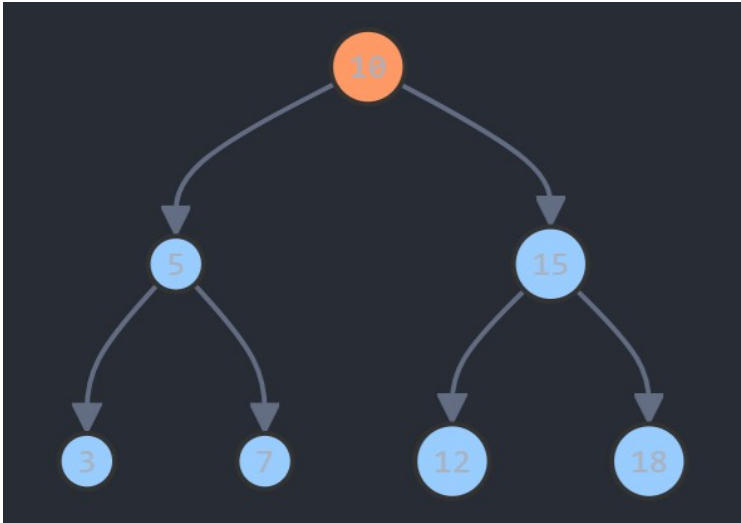


Figure 8.1 : A Binary Tree

TERMINOLOGY

- Node: The primary element that references child nodes
- Root: The main node in the tree without any parent
- Leaf: A node without children.
- Height: The longest path's length from the root node to a leaf node

TYPES OF BINARY TREES

Full Binary Tree

A full binary tree has zero or two child nodes. It is sometimes also known as a proper Binary Tree.

Complete Binary Tree

A complete binary tree has nodes at all levels except the last level.

Perfect Binary Tree

In a perfect binary tree all internal nodes have two child nodes and leaf nodes are at the same level.

TRAVERSING BINARY TREES

We can visit the nodes of the tree in in-order, pre-order, and post-order way.

IN-ORDER TRAVERSAL

The order of traversal is left node first followed by root node and then right node.

```

def in_order_traversal(node):
    if node:
        in_order_traversal(node.left)
        print(node.data)
        in_order_traversal(node.right)
  
```

Example usage

```

in_order_traversal(root)
'''
  
```

In-order traversal is depicted in the following diagram

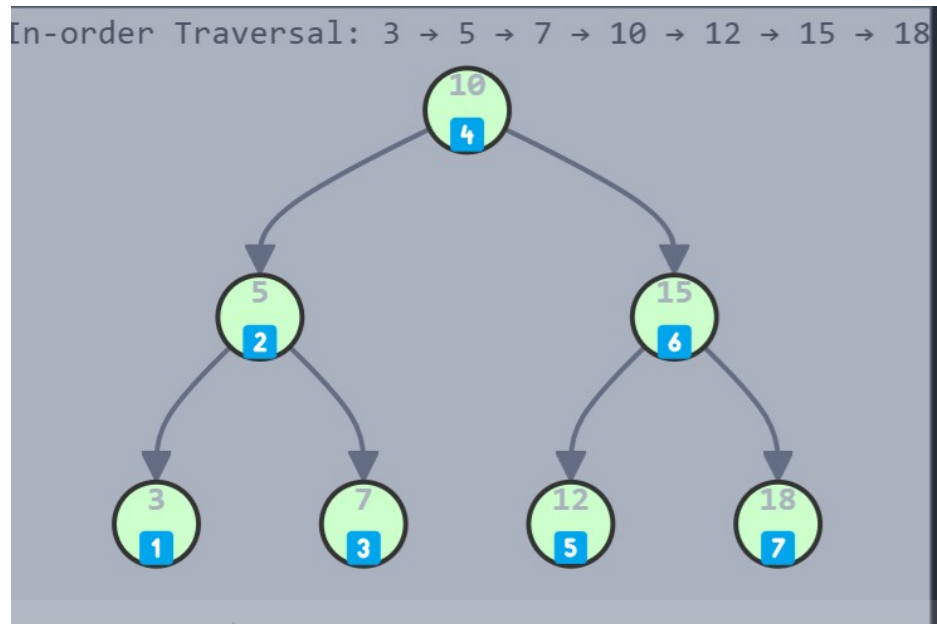


Figure 8.2 :Inorder traversal of a Binary Tree

PRE-ORDER TRAVERSAL

Root node is visited first followed by its children in pre-order traversal.

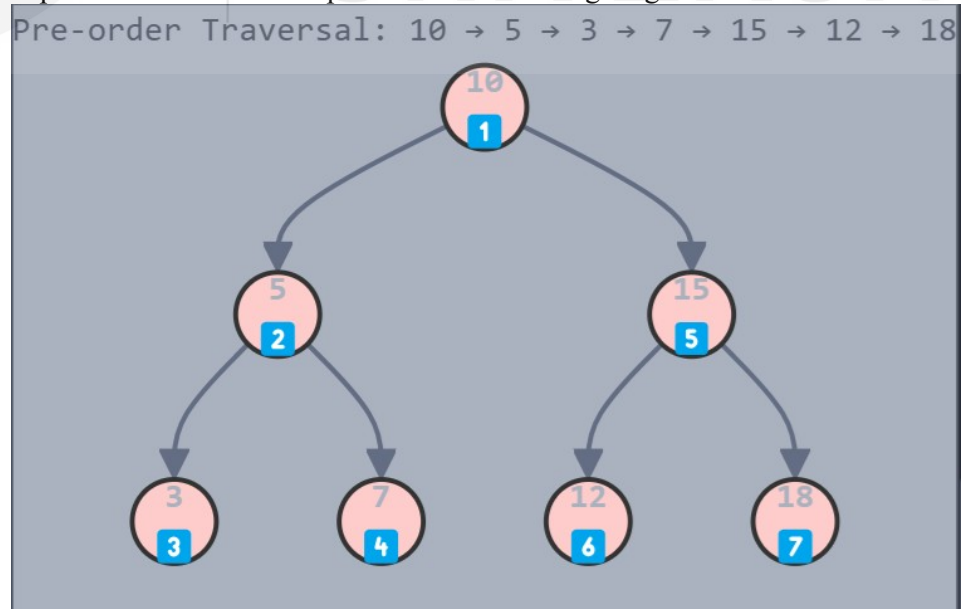
```

def pre_order_traversal(node):
    if node:
        print(node.data)
        pre_order_traversal(node.left)
        pre_order_traversal(node.right)
  
```

```

# Example usage
pre_order_traversal(root)
'''
  
```

A pre-order traversal is depicted in the following diagram

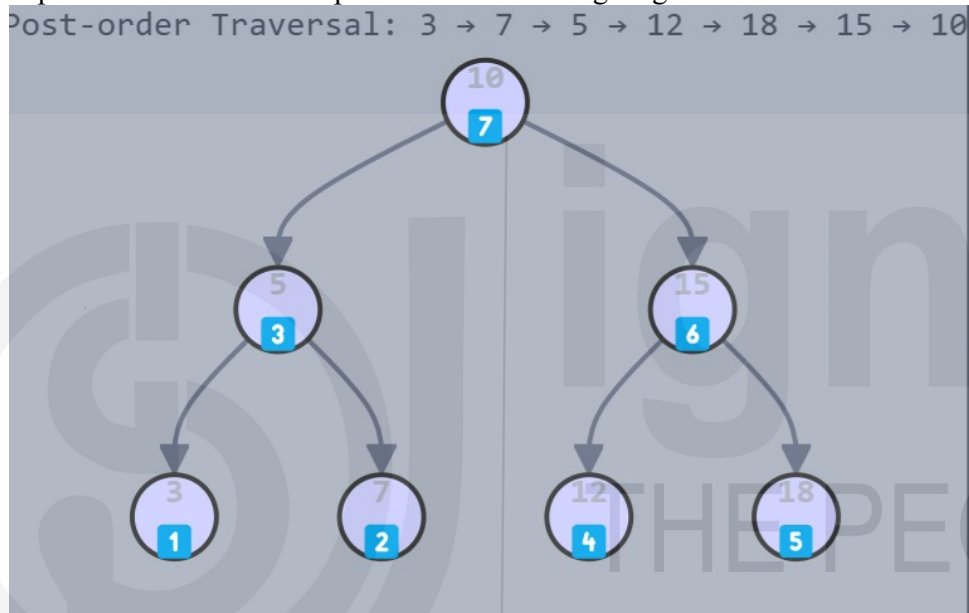


The child nodes are visited before the root node in post-order traversal.

```
def post_order_traversal(node):  
    if node:  
        post_order_traversal(node.left)  
        post_order_traversal(node.right)  
        print(node.data)
```

```
# Example usage  
post_order_traversal(root)  
...
```

A post order traversal is depicted in the following diagram



PRACTICAL USE CASES OF BINARY TREES

Binary Trees find applications in various domains:

1. File System Organization: Directory structures in file systems
2. Expression Evaluation: Mathematical expression parsing
3. Decision Trees: In machine learning for classification
4. Huffman Coding: Data compression algorithms
5. Database Indexing: B-trees and their variants

Let's implement a practical example of using Binary Trees for expression evaluation:

```
class ExpressionTree:  
    def __init__(self, data):  
        self.data = data  
        self.left = None  
        self.right = None  
  
    def evaluate(self):  
        # Leaf node (operand)  
        if not self.left and not self.right:  
            return float(self.data)
```

```

        # Evaluate left and right subtrees
        left_val = self.left.evaluate()
        right_val = self.right.evaluate()

        # Apply operator
        if self.data == '+': return left_val + right_val
        if self.data == '-': return left_val - right_val
        if self.data == '*': return left_val * right_val
        if self.data == '/': return left_val / right_val

```

BEST PRACTICES

1. When implementing Binary Trees, consider these best practices:
2. Always check for empty tree conditions
3. Use recursive solutions for tree traversal when tree depth is reasonable
4. Implement iterative solutions for very deep trees to avoid stack overflow
5. Keep track of tree balance to maintain optimal performance
6. Use appropriate tree variants based on the use case (BST for searching, AVL for balanced operations)

Time complexity for basic operations:

- Insertion: $O(n)$ for a basic binary tree
- Searching: $O(n)$ for a basic binary tree
- Traversal: $O(n)$ for all methods

Binary trees are an essential data structure that provides an efficient way to organize and manage hierarchical data. By understanding their implementation and traversal methods in Python, you can effectively utilize them across various applications ranging from search algorithms to expression parsing. Mastering binary trees will enhance your problem-solving capabilities as you progress in your programming journey.

8.4 Implementing Trees

IMPLEMENTING GENERAL TREES IN PYTHON

To implement a general tree in Python, we can create a 'Node' class to represent each node in the tree. Each node will hold data and a list of its children.

```

class Node:
    def __init__(self, data):
        self.data = data
        self.children = []

    def add_child(self, child_node):
        self.children.append(child_node)

    def __repr__(self):
        return f"Node({self.data})"
...

```

To create a general tree, we can instantiate nodes and establish parent-child relationships using the `add\_child` method.

```
# Create nodes
root = Node("CEO")
cto = Node("CTO")
cfo = Node("CFO")
dev1 = Node("Developer 1")
dev2 = Node("Developer 2")

# Build the tree
root.add_child(cto)
root.add_child(cfo)
cto.add_child(dev1)
cto.add_child(dev2)
...
```

IMPLEMENTING BINARY TREES IN PYTHON

To implement a binary tree in Python, we can create a `Node` class to represent each node in the tree.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None
    ...
```

CREATING A BINARY TREE

To create a binary tree, we can instantiate nodes and establish parent-child relationships.

```
# Create nodes
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
...
```

BINARY TREE OPERATIONS

Let us implement the basic operations that can be performed on a Binary Tree:

```
class BinaryTree:
    def __init__(self):
        self.root = None # Initialize an empty tree
```

```

def insert(self, data):
    if not self.root:
        self.root = Node(data)
        return

    # Use queue for level order insertion
    queue = [self.root]
    while queue:
        node = queue.pop(0)

        if not node.left:
            node.left = Node(data)
            return
        else:
            queue.append(node.left)

        if not node.right:
            node.right = Node(data)
            return
        else:
            queue.append(node.right)

```

8.5 Tree Traversal Algorithms

We shall look at various traversal algorithms in this section.

TREE TRAVERSAL: DEFINITION AND IMPORTANCE

Tree traversal refers to the process of visiting all the nodes in a tree data structure in a systematic manner. The order in which nodes are visited can significantly affect the performance of operations such as searching and sorting.

TYPES OF TREE TRAVERSAL

The main types of tree traversal algorithms are:

- Depth-First Traversal: Visits nodes along a branch before backtracking.
 - Pre-order Traversal
 - In-order Traversal
 - Post-order Traversal

- Breadth-First Traversal: In this method, we visit the nodes at current depth level before visiting the nodes at next level.

IMPLEMENTING TREE TRAVERSAL ALGORITHMS IN PYTHON

To implement tree traversal algorithms, we will first define a binary tree structure using a 'Node' class.

```

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

```

PRE-ORDER TRAVERSAL

In pre-order traversal, root node is visited first followed by left subtree and then right subtree.

```
def pre_order_traversal(node):
    if node:
        print(node.data)
        pre_order_traversal(node.left)
        pre_order_traversal(node.right)

# Example usage
root = Node(1)
root.left = Node(2)
root.right = Node(3)
pre_order_traversal(root) # Output: 1 2 3
'''
```

IN-ORDER TRAVERSAL

In in-order traversal, we first visit left subtree then root node followed by right subtree.

```
def in_order_traversal(node):
    if node:
        in_order_traversal(node.left)
        print(node.data)
        in_order_traversal(node.right)

# Example usage
in_order_traversal(root) # Output: 2 1 3
'''
```

POST-ORDER TRAVERSAL

In post-order traversal, the nodes are visited as follows: left subtree, right subtree, root.

```
def post_order_traversal(node):
    if node:
        post_order_traversal(node.left)
        post_order_traversal(node.right)
        print(node.data)

# Example usage
post_order_traversal(root) # Output: 2 3 1
'''
```

PRACTICAL USE CASES OF TREE TRAVERSAL ALGORITHMS

Tree traversal algorithms have numerous practical applications:

- Expression Evaluation: In compilers, expression trees are traversed to evaluate mathematical expressions.
- File System Navigation: Traversing directory structures to list files and folders.
- Database Indexing: Efficiently searching through hierarchical data structures used in databases.

BEST PRACTICES

When implementing tree traversal algorithms, consider these best practices:

- Use Recursion Judiciously: While recursive implementations are often simpler and more readable, be cautious of stack overflow errors with deep trees.
- Optimize for Performance: For large trees, consider iterative approaches where applicable to avoid excessive memory usage.
- Encapsulation: Keep traversal methods encapsulated within the tree class to maintain clean code architecture.

We can leverage the tree traversal algorithms to access the hierarchical data. Trees are used in various domains such as databases, file systems, and expression evaluation.

8.6 Case Study – An Expression Tree

In computer science, expression trees are a specialized form of binary trees used to represent expressions. Each internal node corresponds to an operator, while the leaf nodes represent operands. We can efficiently evaluate expressions and is widely used in compilers and interpreters. In this chapter, we will explore the concept of expression trees, their implementation in Python, and their practical applications.

EXPRESSION TREES: DEFINITION AND STRUCTURE

An expression tree is a binary tree where:

- Each internal node represents an operator (e.g., +, -, *, /).
- Each leaf node represents an operand (e.g., constants or variables).

TERMINOLOGY

- Operator: A symbol that represents an operation (e.g., addition).
- Operand: A value on which the operation is performed.
- Subtree: A tree formed by a node and its descendants.

IMPLEMENTING AN EXPRESSION TREE IN PYTHON

To implement an expression tree in Python, we will define a 'Node' class to represent each node in the tree.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
'''
```

CREATING AN EXPRESSION TREE

Linked List

We can create an expression tree for the expression $((a + b) * (c - d))$.
A tree would look like this:

```
'''
    *
   /\
  + -
 /\ /\
a b c d
'''
```

The implementation can be done as follows:

```
def create_expression_tree():
    root = Node('*')
    root.left = Node('+')
    root.right = Node('-')
    root.left.left = Node('a')
    root.left.right = Node('b')
    root.right.left = Node('c')
    root.right.right = Node('d')
    return root

expression_tree_root = create_expression_tree()
'''
```

EVALUATING AN EXPRESSION TREE

To evaluate the expression represented by the tree, we can use a recursive function that processes each node based on whether it is an operator or an operand.

```
def evaluate_expression_tree(node):
    if node is None:
        return 0

    # leaf node (operand)
    if node.left is None and node.right is None:
        return float(node.value) # Convert to float for arithmetic operations

    # Evaluate left and right subtrees
    left_value = evaluate_expression_tree(node.left)
    right_value = evaluate_expression_tree(node.right)

    # Perform the operation at this node
    if node.value == '+':
        return left_value + right_value
    elif node.value == '-':
        return left_value - right_value
    elif node.value == '*':
```

```

        return left_value * right_value
    elif node.value == '/':
        return left_value / right_value

# Example usage
result = evaluate_expression_tree(expression_tree_root)
print(f"The result of the expression is: {result}")

```

PRACTICAL USE CASES OF EXPRESSION TREES

Expression trees have several practical applications:

- Compilers: Used for parsing and evaluating expressions in programming languages.
- Mathematical Expression Evaluation: Facilitates efficient computation of complex mathematical expressions.
- Symbolic Computation: Enables manipulation and simplification of mathematical expressions.

BEST PRACTICES

For expression trees, consider these best practices:

- Use Clear Naming Conventions: Ensure that variable names reflect their purpose to enhance code readability.
- Error Handling: Implement error handling for cases such as division by zero or invalid operations.
- Documentation: Comment your code adequately to explain complex logic or algorithms used.

Expression trees provide a powerful way to represent and evaluate mathematical expressions efficiently.

Check Your Progress 1

1. Question: Which of the following is NOT a type of tree traversal? a) Pre-order traversal b) In-order traversal c) Sequential traversal d) Post-order traversal
 - a. Answer: c) Sequential traversal Explanation: The standard tree traversal methods are pre-order, in-order, post-order, and level-order (breadth-first). Sequential traversal is not a standard tree traversal method.
2. Question: Give the highest possible nodes at level 3 (assuming root is at level 0)? a) 4 b) 8 c) 16 d) 2
 - a. Answer: b) 8 Explanation: At any level n the highest number of nodes is 2^n . For level 3, this is $2^3 = 8$ nodes.
3. Question: A binary tree with 10 nodes has how many NULL links (empty child references)? a) 9 b) 10 c) 11 d) 12
 - a. Answer: c) 11 Explanation: Binary tree has two child nodes. For n nodes, there are $2n$ references total. The number of NULL references is $(2n - (n-1))$, where $(n-1)$ is the number of actual parent-child links. For 10 nodes: $2(10) - 9 = 11$ NULL references.

4. Question: Which type of binary tree has all internal nodes containing exactly two children? a) Complete binary tree b) Full binary tree c) Perfect binary tree d) Balanced binary tree Answer: b) Full binary tree
Explanation: A full binary tree is defined as a tree where every node has either 0 or 2 children. All internal nodes must have exactly two children.
5. Question: What is the order of visiting nodes in a pre-order traversal? a) Root, Right, Left b) Left, Root, Right c) Root, Left, Right d) Left, Right, Root
a. Answer: c) Root, Left, Right Explanation: In pre-order traversal, we visit the root node first, then traverse the left subtree, and finally traverse the right subtree.
6. Question: In an expression tree, where are the operands located? a) Only at the root b) Only at leaf nodes c) At any internal node d) Only at the second level
a. Answer: b) Only at leaf nodes Explanation: In an expression tree, operands (numbers or variables) are always located at leaf nodes, while operators (+, -, *, /) are located at internal nodes.
7. Question: What would be the level-order traversal of the following binary tree?
Copy
- ```

 1
 /\
 2 3
 /\
 4 5

```
- a) 1, 2, 3, 4, 5 b) 1, 3, 2, 5, 4 c) 4, 2, 5, 1, 3 d) 1, 2, 3, 5, 4  
a. Answer: a) 1, 2, 3, 4, 5 Explanation: Level-order traversal visits nodes level by level from left to right. Starting at root (1), then level 1 (2,3), then level 2 (4,5).
8. Question: A complete binary tree with 7 nodes has which height? a) 2 b) 3 c) 4 d) 5  
a. Answer: a) 2 Explanation: In a complete binary tree, all levels except possibly the last are completely filled. With 7 nodes, we can fill level 0 (1 node), level 1 (2 nodes), and partially fill level 2 (4 nodes), giving a height of 2.
9. Question: Which data structure is typically used to implement level-order traversal? a) Stack b) Queue c) Linked List d) Array  
a. Answer: b) Queue Explanation: Level-order traversal uses a queue data structure to keep track of nodes at each level, processing them in FIFO (First In First Out) order.
10. Question: What is the time complexity of inserting a node into a binary tree using level-order insertion? a)  $O(1)$  b)  $O(\log n)$  c)  $O(n)$  d)  $O(n^2)$   
a. Answer: c)  $O(n)$  Explanation: Level-order insertion requires traversing the tree to find the first available position, which in worst case requires visiting all nodes, giving a time complexity of  $O(n)$ .

## 8.7 Summary

Trees are versatile data structures that provide efficient ways to organize hierarchical data. The unit covered:

- General trees and binary trees with their properties and implementations
- Different types of binary trees (full, complete, perfect)

- Tree traversal algorithms (pre-order, in-order, post-order, level-order)
- Expression trees and their evaluation
- Practical applications in file systems, databases, and expression evaluation

---

## 8.8 Further Readings

---

- "Data Structures and Algorithms in Python" by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser
- GeeksforGeeks: Tree Data Structure documentation
- "Python Data Structures and Algorithms" by Benjamin Baka

